



Informe Tarea 3

Nombre: Felipe Robino Schutze

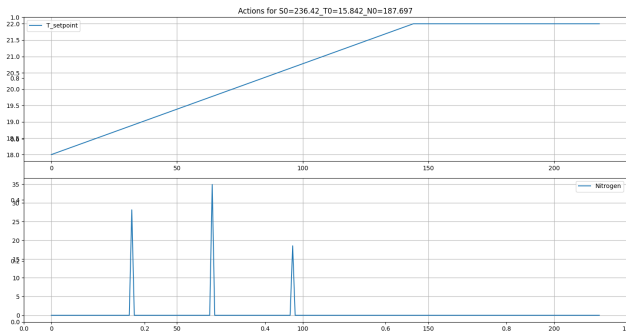
Pregunta 2:

(a)

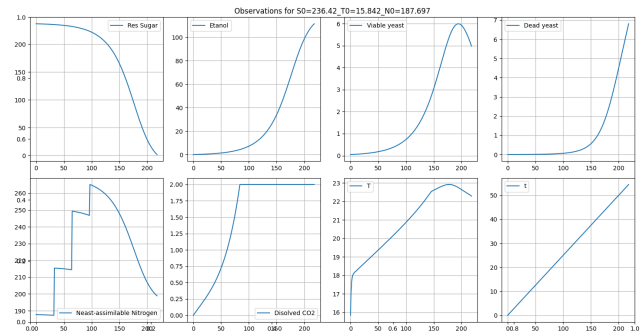
Se adaptó el código `try_expert_controller.py`, para que guardara las figuras y scores de cada intento. Los scores de ambos vinos en las dos condiciones iniciales fueron:

$$[S0 = 236.42, T0 = 15.842, N0 = 187.697] \Rightarrow Score = 6.221185259876885$$

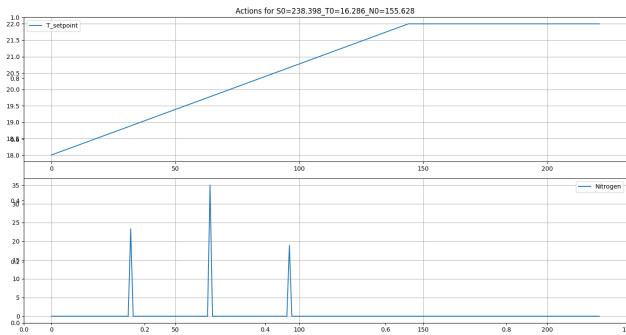
$$[S0 = 238.398, T0 = 16.286, N0 = 155.628] \Rightarrow Score = 6.313162185801326$$



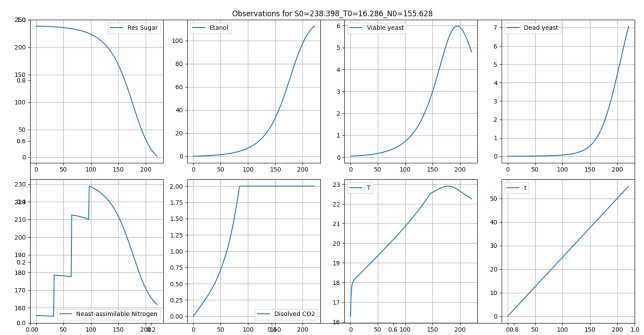
(a) Acciones del controlador experto para el primer set de condiciones iniciales.



(b) Observaciones de estado para el primer set de condiciones iniciales.



(a) Acciones del controlador experto para el segundo set de condiciones iniciales.



(b) Observaciones de estado para el segundo set de condiciones iniciales.

Observando las decisiones del controlador experto, se pueden apreciar patrones generales, que son validables al analizar el script `expert_controller.py`. Controla ambas variables manipuladas de manera independiente. Respecto a la temperatura, sigue el patrón de:

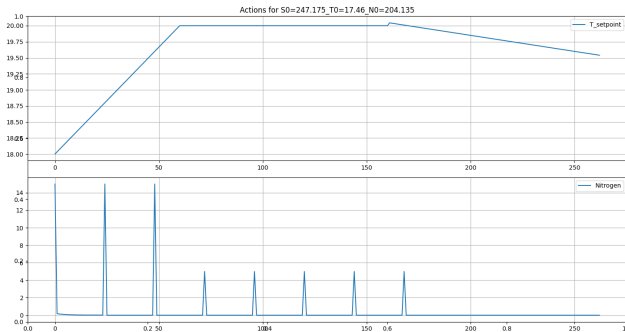
1. Calentar desde 18°C hasta 22°C linealmente, hasta alcanzar este segundo setpoint.
2. Mantener estos 22°C (meseta) constantes.
3. No se aprecia en los ejemplos (en el código sí) – una fase final de enfriamiento (200 h - 336 h) donde se disminuye de 22°C a 16°C.

La estrategia experta del nitrógeno es más puntual y precisa, los peaks se pueden apreciar tanto en la acción como en la observación (Neast-assimilable Nitrogen). Las reglas que dominan al experto son:

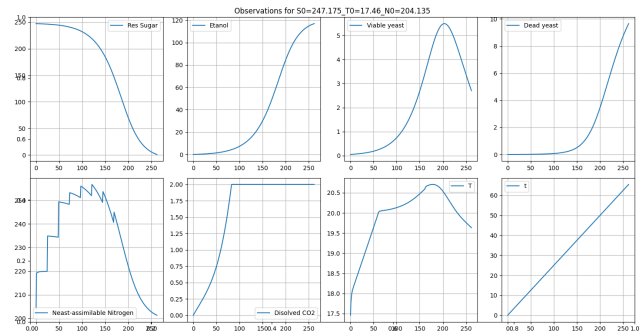
- No agregar nitrógeno cuando la fermentación ya está casi terminada ($S < 10$).
- Agregar una sóla vez nitrógeno de manera temprana ($T \geq 8$ h), con dosis saturada $0.15 \times N0$.
- Después de las 5 horas, si el nitrógeno se encuentra demasiado bajo ($N < 60$), se agrega una dosis de compensación.
- Si hay fermentación muy lenta, se compensa agregando más nitrógeno ($S > 50$ queda mucha azúcar para $t \geq 12$ h)
- Para todas estas acciones hay un buffer (cooldown) que no deja que se agreguen por 8 h nuevas dosis de nitrógeno.

Con esto en mente, pareciera ser que la estrategia experta se centra en hacer un jumpstart de la fermentación temprana (si es que no hay demasiado nitrógeno inicialmente), para luego sólo agregar Nitrógeno si es que se está viendo limitada la fermentación (consumo de azúcar), todo esto siempre limitando la adición y no sobre-dosificando al vino.

A partir de esto, se generó otro controlador experto en el (Código) `expert_controller_v2.py`, que mejora el comportamiento descrito recién. Las mejoras son que en vez de utilizar como parámetro global el tiempo, usa el conversion de azúcar (que es una medida de cuánta azúcar se ha consumido por la fermentación). Además, usa microdosis de nitrógeno más dispersas (en vez de las inyecciones fuertes del otro controlador). Los resultados se pueden ver en la Figura ??, donde se puede apreciar el control distinto que se aplicó. El score del vino generado por este controlador es consistentemente mejor que el otro (en el caso específico es de Score: 7.5116).



(a) Acciones tomadas por el controlador experto mejorado.



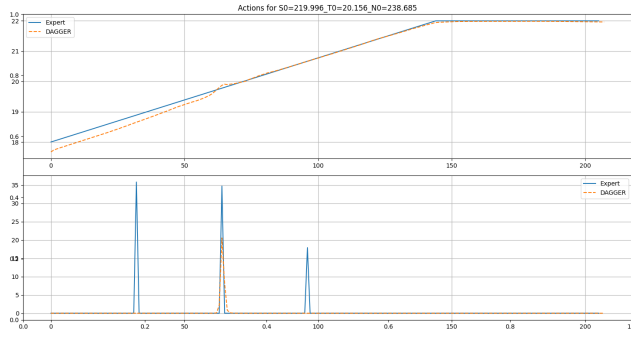
(b) Observaciones de estado paa el controlador experto mejorado.

(b)

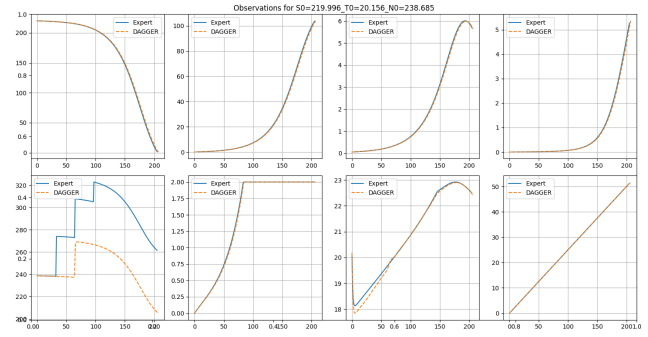
Se completó la implementación de DAGGER en el Código `dagger.py`, usando de base el pseudocódigo de las slides del curso. El código corre el entrenamiento para las distintas trayectorias, y retorna un controlador entrenado a partir del experto, que luego simula desde las mismas condiciones iniciales que el experto para comparar sus comportamientos. Los resultados se pueden apreciar en la Figura ?? y Figura ?. Se puede apreciar cómo el controlador aprende de manera bastante buena el control de temperatura, y el peak más importante de nitrógeno, pero algunas finezas del control se pierden (peaks más pequeños de nitrógeno). Esto lleva a que las trayectorias y scores del controlador DAGGER sean ligeramente distintos, aunque a veces esas variaciones llevan a mejores scores, en particular:

$[S0 = 219.232, T0 = 21.996, N0 = 224.977] \Rightarrow \text{Score experto} = 5.6923, \text{Score DAGGER} = 5.5864$

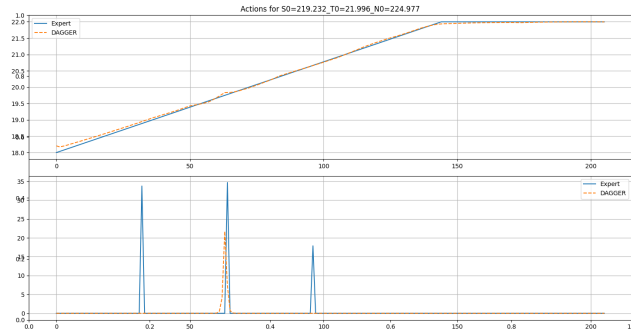
$[S0 = 219.996, T0 = 20.156, N0 = 238.685] \Rightarrow \text{Score experto} = 5.7768, \text{Score DAGGER} = 5.8188$



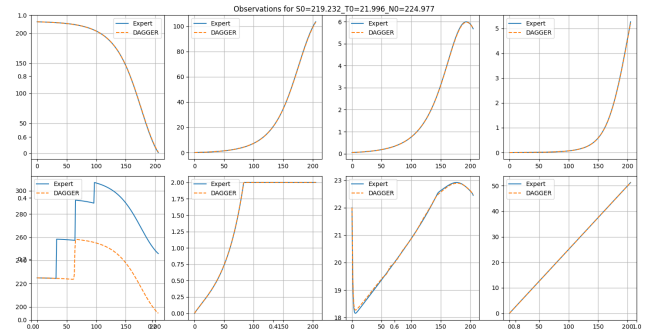
(a) Comparación de la acción del controlador experto vs el entrenado con DAGGER para las primeras condiciones iniciales.



(b) Comparación del estado (observaciones) del controlador experto vs el entrenado con DAGGER para las segundas condiciones iniciales.



(a) Comparación de la acción del controlador experto vs el entrenado con DAGGER para las primeras condiciones iniciales.



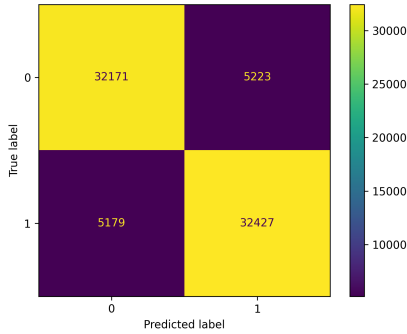
(b) Comparación del estado (observaciones) del controlador experto vs el entrenado con DAGGER para las segundas condiciones iniciales.

(c)

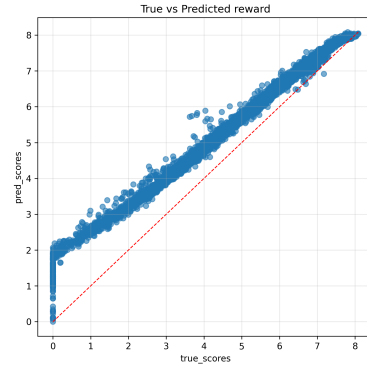
Se adaptó el Código `fit_reward.py` para que haga lo siguiente en orden:

- Cargar el dataset `comparisons.pkl`, y dividirlo en 70 % train, 15 % validate, y 15 % test (luego normalizarlo).
- Cargar el modelo (`RewardNet`), si se elige entrenar, usar `BCEWithLogitsLoss` como pérdida y entrenar por 100 épocas, guardar el mejor modelo.
- Usando el dataset original, comparar las predicciones de la red entrenada con las comparisons reales, y generar la matriz de confusión.
- Finalmente, usando `get_wine_quality_score(profile)`, comparar el score asignado por la red vs. el score real.

La matriz de confusión obtenida se puede apreciar en la Figura 6a, donde se puede ver que la **accuracy** de la red es bastante buena (0.8613). En términos de la predicción de scores realizado por la red, se debe hacer primero un reescalamiento (dado que la red tiene escala libre y no del 0 al 8 como el database). Una vez hecho esto, queda la curva de la Código , donde se puede apreciar la buena calidad de la estimación realizada por la red. Cabe notar que la red tiende a sobreestimar levemente el score de los vinos (para scores de 0 reales varía bastante, y en general tiende a valorar los vinos de peor calidad un poco más altos). El valor de R^2 para el fit de scores es de $R^2 = 0.8385$, lo que indica una alta correlación entre el score asignado por la red y el verdadero (la red entrenada es un buen estimador y clasificador).



(a) Matriz de confusión obtenida para la clasificación de la red entrenada.



(b) Comparación del score predicho por la red entrenada y el score real de perfiles únicos de vino (`reward_fit`).

(d)

Se generó el Código `p2d.py`, que implementa el script `TD3.py` de la pregunta 3 para el environment de vinos. El script entrena el agente TD3 usando como punto de partida al agente DAGGER ya entrenado, usando como crítico a la red recién entrenada. El reward que se le da es sparse (reward directo por el score asignado al vino al final por la red crítica), y se le agregó un reward por terminar la fermentación ($S < 1.0$) para guiar un poco más al actor. Una vez entrenado, se comparan las trayectorias del mejor actor TD3 entrenado, junto a la DAGGER y el controlador experto. Los resultados de las curvas de entrenamiento se pueden apreciar en la Figura 7

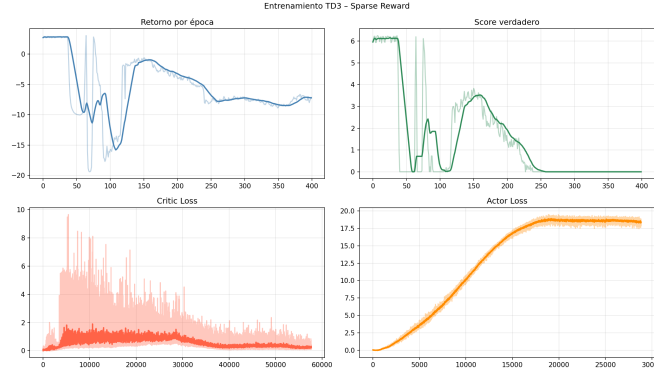
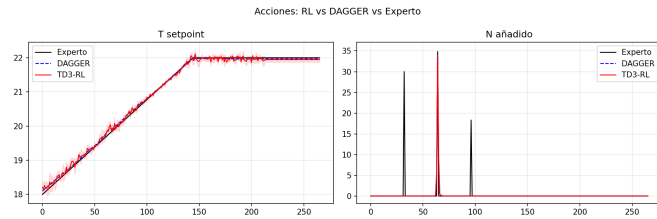
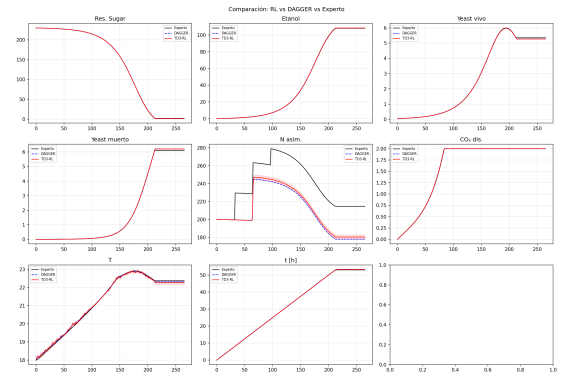


Figura 7: Curvas de entrenamiento para TD3 implementado en WineFermentationEnv.

Se puede apreciar en las curvas de entrenamiento cómo el reward sparse lleva a mal desempeño del actor. Es probable que esto se debe a que en el replay buffer, se guardan demasiadas trayectorias con $\text{reward} = 0.0$, lo que lleva a gradientes mal definidos y una mala dirección para el actor y crítico. Al final, el mejor modelo termina siendo de los más cercanos al DAGGER con que se inicializa (la exploración tiene mala dirección, por lo que a medida que se avanza se llega a mal performance del actor).



(a) Comparación de las acciones de todos los controladores (Experto, DAGGER, TD3).



(b) Comparación del estado (observaciones) de todos los controladores (Experto, DAGGER, TD3).

Los scores alcanzados por los actores entrenados se pueden apreciar en la Figura 9, donde se puede apreciar que el controlador TD3 es básicamente igual a DAGGER (el entrenamiento no mejora sobre este controlador).

Las complicaciones de entrenar un modelo con rewards únicamente al final del episodio son claras. Debido a que se cuenta con un replay buffer sobre el cuál el crítico aprende, y que el proceso es sensible

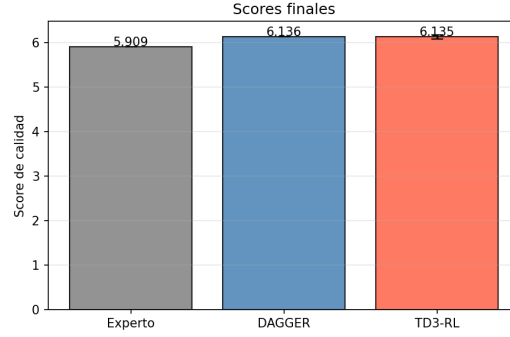


Figura 9: Scores promedio alcanzados por los mejores actores de cada controlador.

(matar la fermentación lleva a $\text{rewards} = 0.0$), se llena el buffer de trayectorias donde no hay reward, y las pocas donde sí hay reward pasan a definir completamente el gradiente (bootstrapping). Ésto lleva a que el gradiente se defina mal, lo que en turno lleva al actor a explorar zonas no deseables debido a bootstrapping de los rewards que sí existen. Con esto, podemos ver en las curvas de entrenamiento cómo una vez que pasa el umbral de `update_after`, el actor tiene una directriz que lleva a catastrophic forgetting y mal rendimiento de la red. Una posible forma de solucionarlo, obviando la posibilidad de agregar rewards intermedios, es llenar el buffer de transiciones generadas por el experto. Esto lleva a que una vez que se pase el umbral `update_after`, el crítico y actor tengan gradientes bien definidos, que evitarían el colapso del entrenamiento (la dirección del gradiente es buena y definida). Otras alternativas a esto son implementar Hindsight Experience Replay, que sigue la lógica de reinterpretar episodios con reward 0.0 como si el objetivo hubiese sido que sí logró fermentar, lo que garantiza una señal no nula para el gradiente.

Pregunta 3:

(a)

Se hicieron cambios mínimos para guardar el modelo mejor entrenado por TD3, además de guardado de imágenes en TD3 y LQR. Al correr la simulación de QuadRotor con el controlador LQR, se obtiene un rendimiento bastante bueno (estable y sigue relativamente bien la referencia), pero que tiene error residual permanente y notorio. Esto se puede apreciar en la Figura 10, donde se ve que hay un constante error de seguimiento respecto a la referencia, pero en la mayor parte sigue de manera aproximada a la referencia. En el caso del controlador entrenado con TD3, el caso es muy distinto: se pierde drásticamente el seguimiento de la referencia, lo que se puede apreciar claramente en la Figura 11

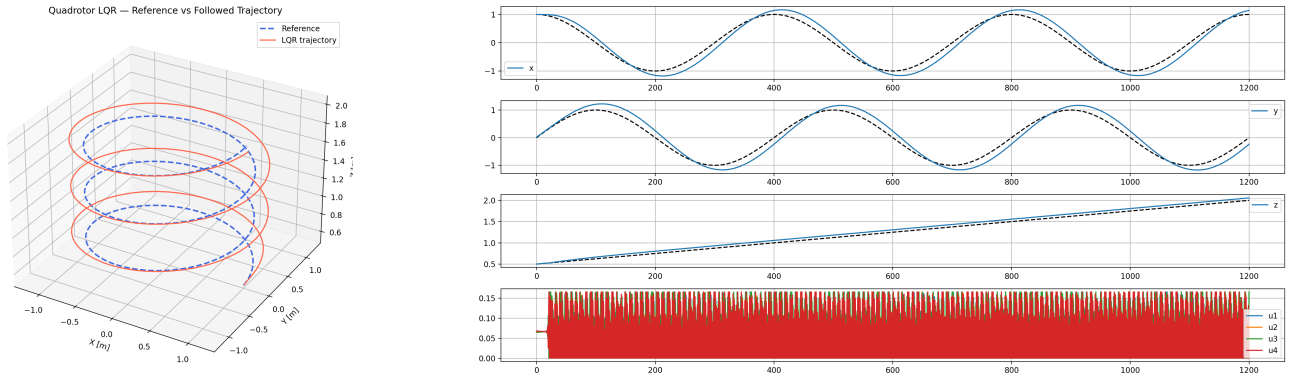


Figura 10: Trayectoria seguida por el dron y referencia (izquierda), y seguimiento de referencias internas (x, y, z) y acciones u_1, u_2, u_3, u_4 en el tiempo (derecha) para el controlador LQR.

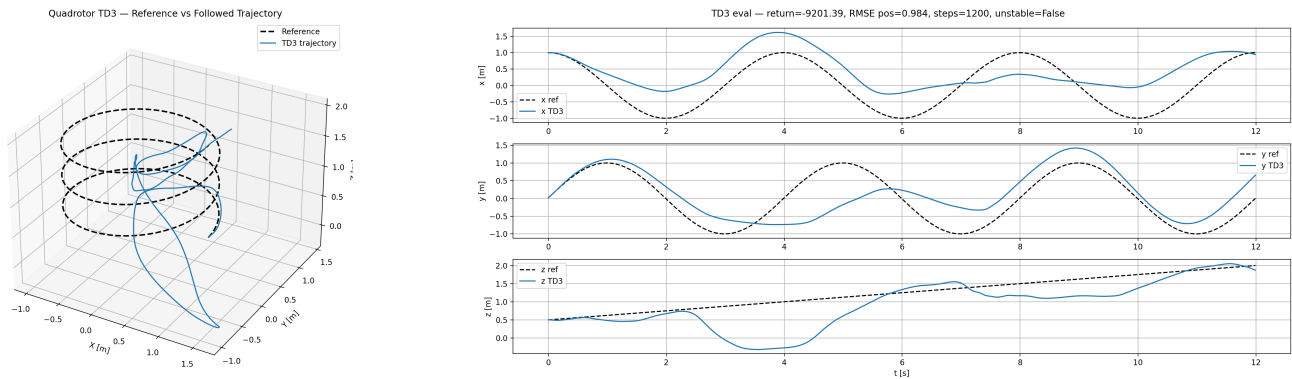


Figura 11: Trayectoria seguida por el dron y referencia (izquierda), y seguimiento de referencias internas (x, y, z) en el tiempo (derecha) para el controlador entrenado con TD3.

El retorno acumulado por durante el entrenamiento se puede apreciar en la Figura 12. Se puede apreciar claramente cómo el entrenamiento no logra realmente converger a una buena política, pese a pasar por alrededor de 1800 episodios de entrenamiento. La gran dificultad que TD3 tiene para aprender a controlar el sistema, es que al ser un algoritmo sin modelo (model-free), debe aprender todas las no-linealidades, alto nivel de acoplamiento, e interacciones complejas del sistema QuadRotor mediante sólo exploración. Esto lleva a aprendizaje muy lento, debido a la alta complejidad de las relaciones que debe capturar. Además, dada la alta inestabilidad del sistema (intrínsecamente inestable en lazo abierto), en los primeros episodios de entrenamiento el agente genera muchas trayectorias aleatorias que casi siempre

llegan a violar las condiciones de estabilidad, terminando estos episodios prematuramente (ver primeros ≈ 600 episodios en la Figura 12 arriba, izquierda). Esto tiene el efecto de llenar el replay buffer de transiciones de episodios fallidos, lo que sesga y empeora el aprendizaje, y calculando el gradiente sobre estas malas y no-representativas trayectorias.

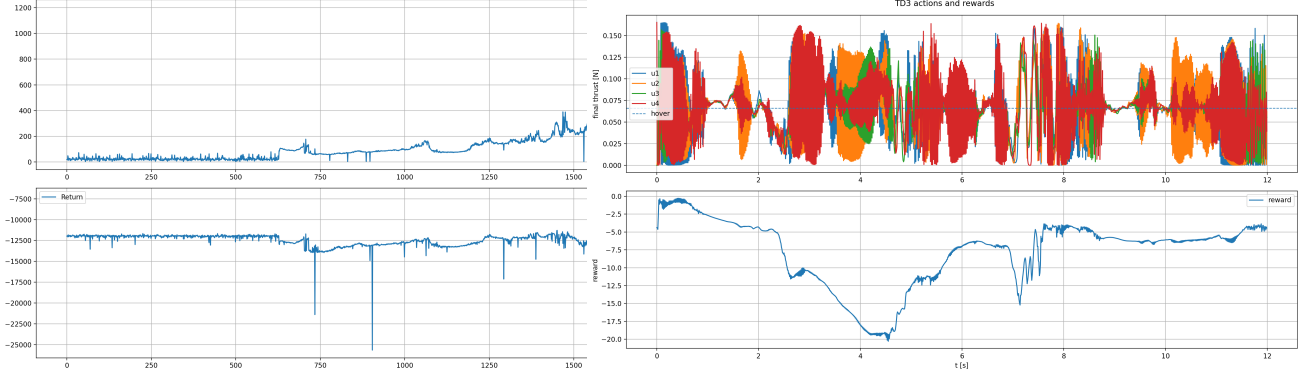


Figura 12: Retorno acumulado (abajo) y largo del episodio (arriba) vs episodio de entrenamiento (izquierda), reward del episodio del mejor controlador TD3 entrenado (reward vs tiempo, derecha).

(b)

Se generó el Código a partir del script de TD3 y el de LQR. En este código, se implementa un controlador residual usando de base el LQR, donde el agente TD3 sólo se entrena sobre la acción residual tomada (transición s_t, u_t, s_{t+1}, r_t). La función de rewards para el agente castiga por igual todos los errores en x, y, z , además de un pequeño término de castigo para acciones muy grandes. Los resultados de la trayectoria generada por el controlador residual se pueden apreciar en la Figura 13, donde se puede ver cómo el controlador residual mejora significativamente el error de seguimiento de variables y aproxima mucho más la trayectoria a la referencia (también se puede ver reflejado esto en el RMSE de posición, que con residual RMSE=0.100, con sólo LQR RMSE = 0.1585). Las curvas de entrenamiento se pueden ver en la (Figura 14).

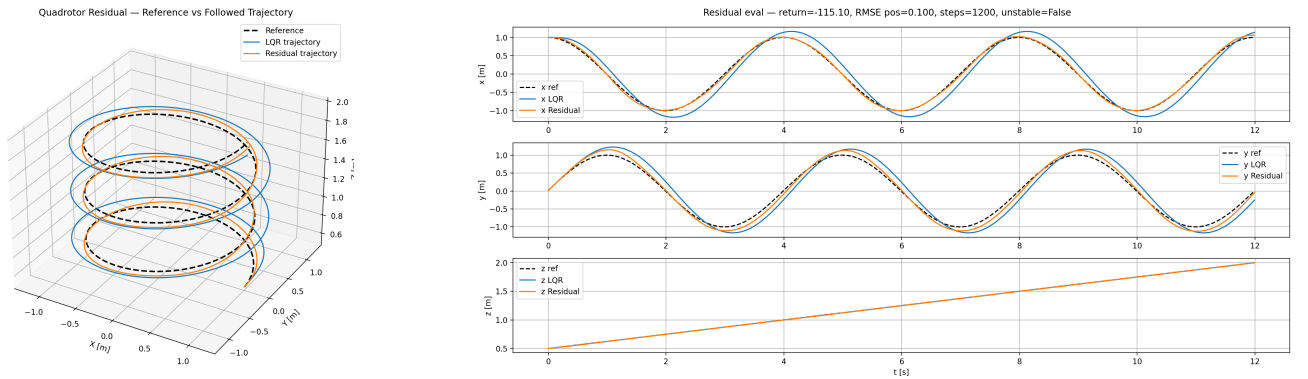


Figura 13: Trayectoria seguida por el controlador LQR base y el residual, junto con el error de seguimiento de ambos respecto a las referencias en x, y, z .

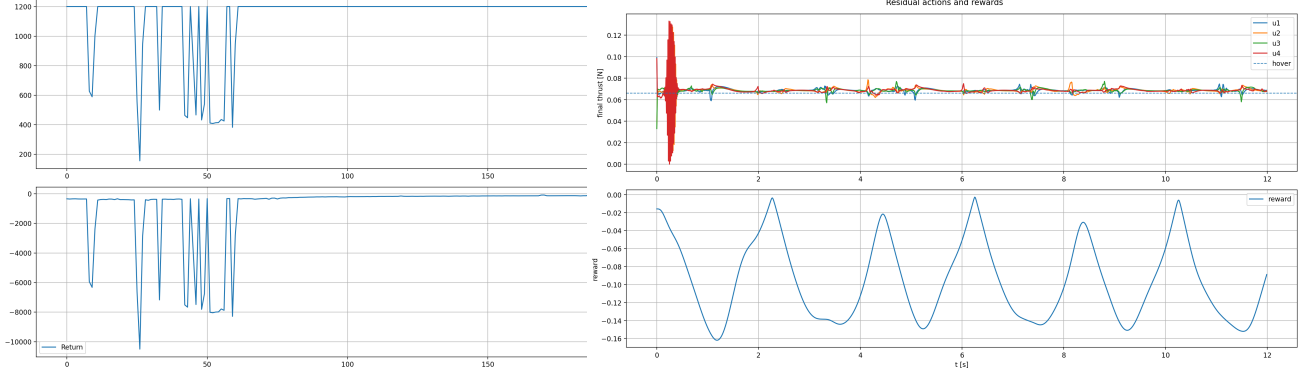


Figura 14: Retorno acumulado (abajo) y largo del episodio (arriba) vs episodio de entrenamiento (izquierda), reward del episodio del mejor controlador residual entrenado (reward vs tiempo, derecha).

(c)

Se adaptó el mismo Código , para que dado el input de domain adaptation, se ajusten los parámetros usando `optimize()` de `reps_algorithm.py`. El funcionamiento del DA es casi igual al de DR, pero cada $N = 10$ pasos, se reactualizan los parámetros, usando como batch sobre el cuál se usa `optimize()` una simulación con la política estática (`noise_scale = 0.0`) de tamaño `batch_size = 32`. Esto tiene que ser así para que el DA se calcule sobre el mismo ambiente, y pueda correctamente estimar los próximos parámetros. Los resultados del agente entrenado con DA, comparados con los DR se pueden apreciar en la Figura 15, y las curvas de entrenamiento en la Figura 16. Se puede ver cómo los resultados de DA son bastante similares a los de DR, comparables en casi todas las métricas incluidos el RMSE (DR = 0.102, DA = 0.105). La similitud entre ambos métodos probablemente se debe al hecho de que tanto en DA como en DR se opera sobre las trayectorias que no fallan. Así, DR también en cierta medida está direccionado hacia los parámetros de operación reales u óptimos (los parámetros muy desviados llevan a trayectorias inviables que se descartan, por lo que DR igual está operando en un dominio similar a DA). Es debido a esto probablemente que ambos algoritmos tienen rendimientos similares y casi indiferenciables.

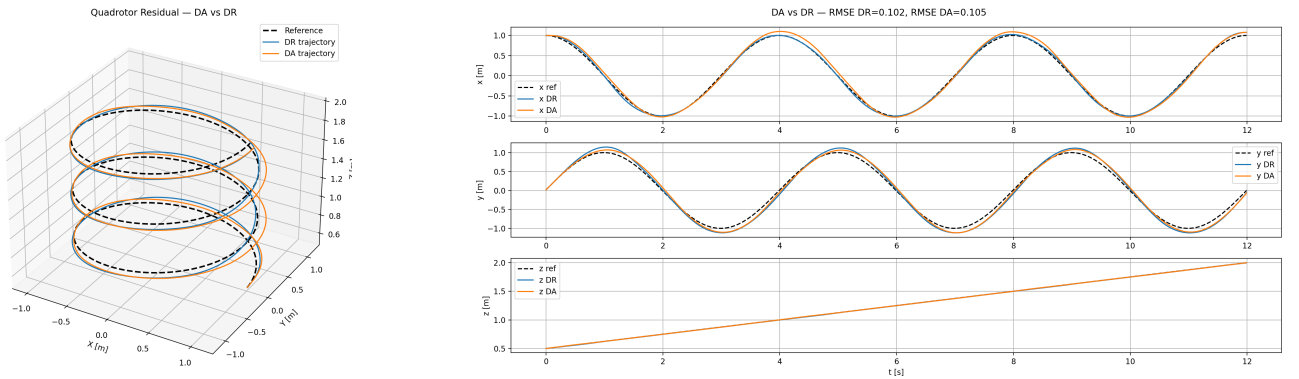


Figura 15: Trayectoria seguida por el controlador LQR base y el residual, junto con el error de seguimiento de ambos respecto a las referencias en x, y, z .

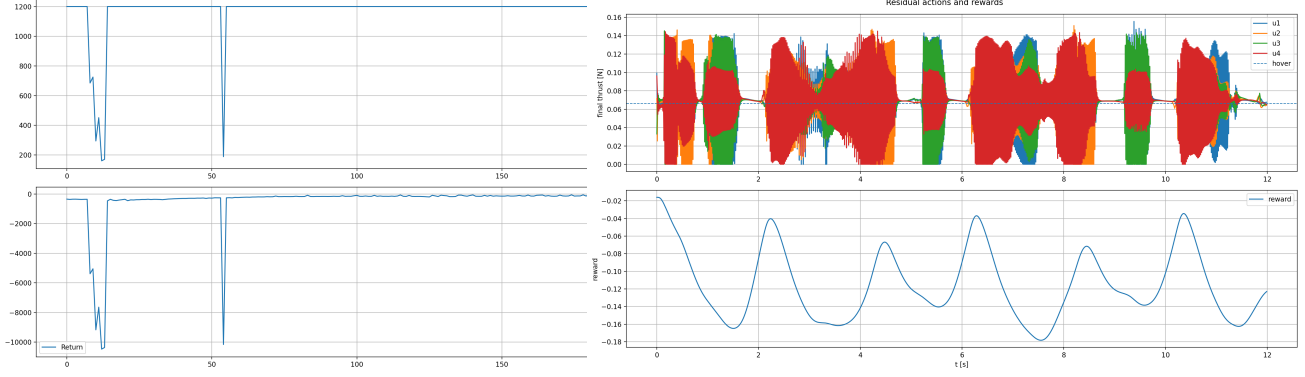


Figura 16: Retorno acumulado (abajo) y largo del episodio (arriba) vs episodio de entrenamiento (izquierda), reward del episodio del mejor controlador residual entrenado con Domain Adaptation (reward vs tiempo, derecha).

(d)

Para este ítem se generó el Código `LQR_tuning.py`, donde se utilizó de base la función `train()` de TD3, con cambios realizados de manera tal que acepte de entrada la observación del LQR, y de acción outputtee un vector de multiplicadores que aumente o disminuya las magnitudes de las matrices Q , R del LQR. Así:

$$a = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_{15} \end{bmatrix}, Q_{new} = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_{11} \end{bmatrix} \cdot Q_{old}, R_{new} = \begin{bmatrix} \lambda_{12} \\ \lambda_{13} \\ \lambda_{14} \\ \lambda_{15} \end{bmatrix} \cdot R_{old}$$

Luego de esta transformación de matrices, se utiliza la nueva función `recalculate_K()` que se implementó en `LQR.py` para obtener mediante Ricatti la matriz de ganancias K . Con esta, se dan pasos de simulación y se calcula el `reward_function_complex` sobre dicha acción del LQR con esa ganancia. Se guarda el mejor modelo de LQR obtenido tras el entrenamiento, que tiene las curvas de entrenamiento de la Figura 17, que se puede ver lo errático que es. Esta forma de la curva de entrenamiento probablemente se debe al alto nivel de desacople que hay entre la función de rewards (calculado sobre la acción tomada por el LQR calculado a partir de los pesos adaptados por el agente) y el agente, lo que lleva a gradientes extraños y aprendizaje un poco aleatorio. Sin embargo, se guarda el mejor modelo encontrado, que tiene un rendimiento superior al del LQR base. Los pesos de las matrices óptimas obtenidas por el agente son:

$$R_{opt} = \text{diag}(0.0948, 0.054, 0.0772, 0.0992)$$

$$Q_{opt} = \text{diag}(16.1975, 1.204, 49.8946, 3.571248, 7.694, 0.2, 0.1879, 4.0154, 0.0174, 0.01, 0.0101, 0.9884)$$

En la función de rewards, se obtuvo como resultado para LQR base -10866.755 , mientras que para el LQR sintonizado por el agente TD3, se obtuvo -6696.926 , una mejora de aproximadamente 40% en el retorno acumulado. Respecto a la trayectoria y el seguimiento de ésta, se puede apreciar en la Figura 18, cómo la sintonización del agente permite mejoras sobre el caso base. El RMSE del LQR base es de 0.15855, mientras que el del LQR sintonizado por TD3 es de 0.1041 [m]. Así, podemos afirmar que la sintonización genera mejoras marginales sobre el controlador base - a pesar de verse limitado por el comportamiento errático del entrenamiento.

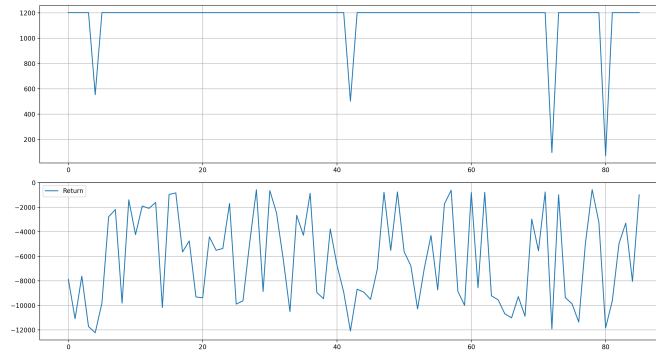


Figura 17: Curva de entrenamiento (retorno acumulado por episodio - abajo, largo del episodio - arriba) para el sintonizado del LQR mediante el agente TD3.

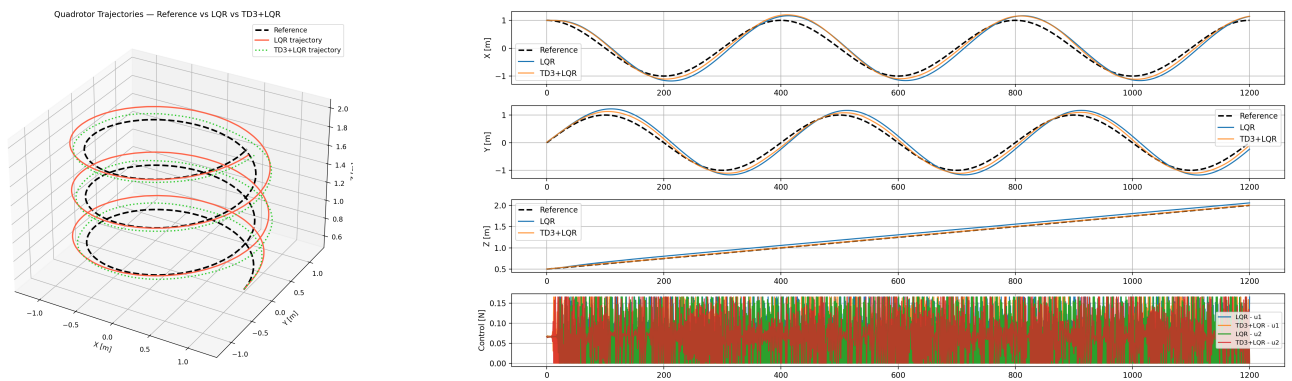


Figura 18: Trayectoria seguida por el controlador LQR base y el LQR sintonizado por el agente TD3, junto con el error de seguimiento de ambos respecto a las referencias en x, y, z .

expert_controller_v2.py

```

1  import numpy as np
2
3  class SmoothController:
4
5      T_START = 18.0
6      T_MID = 20.0
7      T_PEAK = 22.0
8
9      FEED_INTERVAL = 6.0 # H
10     MICRO_DOSE = 5.0
11
12     N_TARGET = 220.0
13     N_MAX_CORRECTION = 8.0
14
15     CONVERSION_STOP = 0.5 # conversion m s de 50 -> no alimentar
16
17     def __init__(self):
18         self.reset()
19
20     def reset(self):
21         self._S0 = None
22         self._last_feed_t = -999.9

```

```

23
24     self._early_rates = []
25     self._S_history = []
26
27     ####
28     ## TEMPERATURA IGUAL PERO M S SUAVE LA CURVA
29
30     def setpoint(self, t):
31         # 18 -> 20
32         if t <= 15:
33             return 18.0 + (20.0 - 18.0) * t / 15.0
34         # mantener 20
35         elif t <= 40:
36             return 20.0
37         # 20 -> 16
38         else:
39             return 20.0 - (4.0 * t - 170.00) / 200.0
40
41     #####
42     # NITR GENO
43
44     # utils
45     def sugar_conversion(self, s):
46         if self._S0 is None:
47             return 0.0
48
49         return np.clip(
50             (self._S0 - s) / max(self._S0, 1e-6),
51             0.0,
52             1.0
53         )
54
55     def dS_dt(self):
56         if len(self._S_history) < 2:
57             return None
58
59         window = self._S_history[-8:]
60
61         if len(window) < 2:
62             return None
63
64         return (window[0] - window[-1]) / (len(window) * 0.25)
65
66
67     # feeding
68     def nitrogen_pol(self, obs, t):
69
70         S, E, X, Xd, N = obs[0], obs[1], obs[2], obs[3], obs[4]
71
72         if self._S0 is None:
73             self._S0 = float(S)
74
75         conv = self.sugar_conversion(S)
76
77         # guardar history
78         self._S_history.append(float(S))
79
80         if len(self._S_history) > 20:
81             self._S_history.pop(0)

```

```

82     rate = self.dS_dt()
83
84
85     # early rates
86     if t <= 24 and rate is not None:
87         self._early_rates.append(rate)
88
89     ### REGLAS
90     # AGREGAR PEAK INICIAL
91     if t <= 15 and (t - self._last_feed_t) >= self.FEED_INTERVAL:
92         self._last_feed_t = t
93         return np.clip(
94             0.15 * self._S0,
95             0.0, 15.0
96         )
97
98     # 1. NO ALIMENTAR M S CUANDO CONVERSI N > 50%
99     if conv > self.CONVERSION_STOP:
100         return 0.0
101
102     dose = 0.0
103
104     # 2. DAR MICRODOSIS CADA X HORAS
105     if (t - self._last_feed_t) >= self.FEED_INTERVAL:
106         dose += self.MICRO_DOSE
107         self._last_feed_t = t
108
109     # 3. MANTENER N CERCA DEL TARGET
110     if N < self.N_TARGET:
111
112         correction = 0.2 * (self.N_TARGET - N)
113
114         dose += np.clip(
115             correction,
116             0.0, self.N_MAX_CORRECTION
117         )
118
119     # 2. DESACELERAR FERMENTACI N
120     if (t > 24 and len(self._early_rates) > 10 and rate is not None):
121         baseline = np.mean(self._early_rates)
122         if baseline > 1e-6:
123             ratio = rate / baseline
124
125             if ratio < 0.70:
126                 dose += 5.0
127             elif ratio < 0.85:
128                 dose += 2.5
129
130     return float(np.clip(dose, 0.0, 15.0))
131
132
133     # call
134     def __call__(self, obs, t):
135         T_sp = self.setpoint(t)
136         N_feed = self.nitrogen_pol(obs, t)
137
138     return np.array([T_sp, N_feed], dtype=np.float32)

```

dagger.py

```
1 import torch
2 import pickle
3 import numpy as np
4 import torch.nn as nn
5 import torch.optim as optim
6 import matplotlib.pyplot as plt
7 from scripts.expert_controller import WineExpertController
8 from scripts.fermentation_env import WineFermentationEnv
9 from scripts.quality_functions import maceration_extraction, get_wine_quality_score
10 import time
11
12 import os
13
14 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
15
16 class Actor(nn.Module):
17     def __init__(self, obs_dim: int, act_dim: int, act_low: np.ndarray, act_high: np.
18         ndarray, hidden: int = 256):
19         super().__init__()
20         self.register_buffer("act_low", torch.as_tensor(act_low, dtype=torch.float32))
21         self.register_buffer("act_high", torch.as_tensor(act_high, dtype=torch.float32))
22
23         self.net = nn.Sequential(
24             nn.Linear(obs_dim, hidden), nn.ReLU(),
25             nn.Linear(hidden, hidden), nn.ReLU(),
26             nn.Dropout(0.3),
27             nn.Linear(hidden, act_dim), nn.Tanh(),
28         )
29
30     def forward(self, obs: torch.Tensor) -> torch.Tensor:
31         x = self.net(obs)
32         x = 0.5 * (x + 1.0)
33         return self.act_low + x * (self.act_high - self.act_low)
34
35
36 class DAGGER:
37     def __init__(self, T_lims=[8, 38], N_lims=[0, 50]):
38         self.T_lims = T_lims
39         self.N_lims = N_lims
40         self.act_lims = [np.array([self.T_lims[0], self.N_lims[0]]), np.array([self.
41             T_lims[1], self.N_lims[1]])]
42
43     def get_normalization_params(self, trajectories):
44         all_states = []
45         for traj in trajectories:
46             states = traj['trajectory']
47             all_states.append(states)
48         all_states = np.concatenate(all_states, axis=0)
49         min_val = all_states.min(axis=0)
50         min_val = np.array([max(elem*0.9, 0) for elem in min_val])
51         max_val = all_states.max(axis=0)
52         max_val = np.array([elem*1.1 for elem in max_val])
53         return min_val, max_val
54
55     def preprocess_data(self, trajectories, ratio=0.8):
```

```

56     all_states = []
57     all_actions = []
58     for traj in trajectories:
59         all_states.append(traj['trajectory'])
60         all_actions.append(traj['actions'])
61
62     all_states = np.concatenate(all_states, axis=0)
63     all_actions = np.concatenate(all_actions, axis=0)
64
65     states_scaled = (all_states - self.state_lims[0]) / (self.state_lims[1] - self.
66         state_lims[0])
67     actions_scaled = (all_actions - self.act_lims[0]) / (self.act_lims[1] - self.
68         act_lims[0])
69
70     indices = np.array([i for i in range(states_scaled.shape[0])])
71     np.random.shuffle(indices)
72
73     states_scaled = states_scaled[indices]
74     actions_scaled = actions_scaled[indices]
75
76     train_states = states_scaled[: int(ratio*len(indices))]
77     train_actions = actions_scaled[: int(ratio*len(indices))]
78     val_states = states_scaled[int(ratio*len(indices)):]
79     val_actions = actions_scaled[int(ratio*len(indices)):]
80     return (train_states, train_actions), (val_states, val_actions)
81
82 def sup_controller(self, trajectories, epochs=500, batch_size=64):
83     self.state_lims = self.get_normalization_params(trajectories)
84     (train_states, train_actions), (val_states, val_actions) = self.preprocess_data(
85         trajectories)
86
87     actor = Actor(obs_dim=len(self.state_lims[0]), act_dim=2,
88         act_low=self.act_lims[0], act_high=self.act_lims[1]).to(
89         device)
90
91     optimizer = optim.Adam(actor.parameters(), lr=1e-3)
92     criterion = nn.MSELoss()
93     patience = 25
94     best_eval = 10000
95     no_better = 0
96
97     train_loss_sup = []
98     val_loss_sup = []
99     for epoch in range(epochs):
100         # Training
101         actor.train()
102         batch = 0
103         train_loss = []
104         while batch < len(train_states):
105             X = torch.from_numpy(train_states[batch: batch + batch_size]).to(device)
106             .float()
107             Y = torch.from_numpy(train_actions[batch: batch + batch_size]).to(device)
108             .float()
109
110             optimizer.zero_grad()
111             Y_pred = actor(X)

```

```

109         Y_pred_norm = (Y_pred - actor.act_low)/(actor.act_high - actor.act_low)
110         loss = criterion(Y_pred_norm, Y)
111         loss.backward()
112         optimizer.step()
113         train_loss.append(loss.item())
114         batch += batch_size
115     train_loss = sum(train_loss) / len(train_loss)
116     train_loss_sup.append(train_loss)
117
118     # Validation
119     val_loss = []
120     with torch.no_grad():
121         actor.eval()
122         batch = 0
123         while batch < len(val_states):
124             X = torch.from_numpy(val_states[batch: batch + batch_size]).to(
125                 device).float()
126             Y = torch.from_numpy(val_actions[batch: batch + batch_size]).to(
127                 device).float()
128             Y_pred = actor(X)
129             Y_pred_norm = (Y_pred - actor.act_low) / (actor.act_high - actor.
130                 act_low)
131             loss = criterion(Y_pred_norm, Y)
132             val_loss.append(loss.item())
133             batch += batch_size
134
135     # Metrics
136     val_loss = sum(val_loss) / len(val_loss)
137     val_loss_sup.append(val_loss)
138     if val_loss < best_eval:
139         best_eval = val_loss
140         no_better = 0
141         print('Best Model Epoch {}|{}-> Validation Loss: {}'.format(epoch,
142             epochs, val_loss))
143         torch.save(actor.state_dict(), 'best_model.pt')
144     else:
145         no_better += 1
146         if no_better >= patience:
147             print('Early stopping')
148             actor.load_state_dict(torch.load('best_model.pt'))
149             return actor
150
151     actor.load_state_dict(torch.load('best_model.pt'))
152     return actor
153
154 def get_action(self, actor, observation):
155     with torch.no_grad():
156         observation = (observation.reshape(1, -1) - self.state_lims[0])/(self.
157             state_lims[1] - self.state_lims[0])
158         action = actor(torch.from_numpy(observation).to(device))
159         return action.cpu().detach().numpy()[0]
160
161 def linear_schedule(self, step: int, total_steps: int, start: float, end: float) ->
162     float:
163     frac = max(0.0, 1.0 - step / float(total_steps))
164     return end + (start - end) * frac
165
166 def collect_trajectory(self, actor, beta):
167     env = WineFermentationEnv()

```



```

162     obs = env.reset()
163     done = False
164
165     expert = WineExpertController()
166
167     states = []
168     expert_actions = []
169     learner_actions = []
170
171     while not done:
172         # acci n del experto
173         a_exp = expert(obs, float(obs[7]))
174
175         # acci n aprendiz
176         a_learn = self.get_action(actor, obs)
177
178         # pol tica DAGGER
179         if np.random.rand() < beta:
180             action = a_exp
181         else:
182             action = a_learn
183
184         states.append(obs)
185         expert_actions.append(a_exp)
186         learner_actions.append(a_learn)
187
188         obs, done, info = env.step(action)
189
190     return {
191         "trajectory": np.array(states),
192         "expert_actions": np.array(expert_actions),
193         "learner_actions": np.array(learner_actions)
194     }
195
196 def run(self, expert_trajectories, n_trajs=100, epochs=100): #COMPLETAR
197     actor = self.sup_controller(expert_trajectories, epochs=epochs)
198
199     dataset = list(expert_trajectories)
200
201     for i in range(n_trajs): # for i = 1 to N do
202         # let pi_i = beta_i * pi_expert + (1-beta_i) * pi_learner
203
204         beta = self.linear_schedule(i, n_trajs, start=1.0, end=0.0)
205         # sample T-step trajs using pi_i
206         # get dataset D_i = {s, pi*(s)} of visited states by pi_i
207         new_traj = self.collect_trajectory(actor, beta)
208
209         # aggregate datasets D = D U D_i
210         dataset.append({
211             "trajectory": new_traj["trajectory"],
212             "actions": new_traj["expert_actions"]
213         })
214
215         # train classifier pi_learner on D
216         actor = self.sup_controller(dataset, epochs=epochs)
217
218     # return best pi_learner on validation
219     return actor
220

```

```

221
222 if __name__ == "__main__":
223     # Training Phase
224     with open('p2/data/expert_trajectories.pkl', 'rb') as f:
225         expert_trajectories = pickle.load(f)
226
227     dagger = DAGGER()
228     actor = dagger.run(expert_trajectories, n_trajs=50)
229
230     path = 'results/p2/dagger'
231     os.makedirs(path, exist_ok=True)
232
233     # mismas condiciones iniciales
234     S0 = np.random.uniform(210, 260)
235     T0 = np.random.uniform(15, 22)
236     N0 = np.random.uniform(150, 280)
237
238     conds = f'S0={S0:.3f}_T0={T0:.3f}_N0={N0:.3f}'
239
240     #####
241     # EXPERTO
242     env_exp = WineFermentationEnv(S0=S0, T0=T0, N0=N0)
243     ctrl = WineExpertController()
244     ctrl.reset()
245
246     obs = env_exp.reset(S0=S0, T0=T0, N0=N0)
247     done = False
248
249     obs_exp = [obs.copy()]
250     act_exp = []
251
252     while not done:
253         t = float(obs[7])
254         action = ctrl(obs, t)
255
256         obs, done, info = env_exp.step(action)
257         act_exp.append(action.copy())
258         obs_exp.append(obs.copy())
259
260     obs_exp = np.array(obs_exp[:-1], dtype=np.float32)
261     act_exp = np.array(act_exp, dtype=np.float32)
262
263     #####
264     # APRENDIZ DAGGER
265     env_dag = WineFermentationEnv(S0=S0, T0=T0, N0=N0)
266
267     obs = env_dag.reset(S0=S0, T0=T0, N0=N0)
268     done = False
269
270     obs_dag = [obs.copy()]
271     act_dag = []
272
273     while not done:
274         action = dagger.get_action(actor, obs)
275
276         obs, done, info = env_dag.step(action)
277         act_dag.append(action.copy())
278         obs_dag.append(obs.copy())
279

```

```

280 obs_dag = np.array(obs_dag[:-1], dtype=np.float32)
281 act_dag = np.array(act_dag, dtype=np.float32)
282
283
284 #####
285 # Scores
286 traj_exp = env_exp.get_trajectory()
287 traj_dag = env_dag.get_trajectory()
288
289 score_exp = get_wine_quality_score(maceration_extraction(traj_exp))
290 score_dag = get_wine_quality_score(maceration_extraction(traj_dag))
291
292 print(f"Score_Expert:_{score_exp}")
293 print(f"Score_DAGGER:_{score_dag}")
294
295 np.save(os.path.join(path, 'score_exp_' + conds + '.numpy'), np.array(score_exp))
296 np.save(os.path.join(path, 'score_dag_' + conds + '.numpy'), np.array(score_dag))
297
298 #####
299 ### Plots
300 names = [
301     'Res_Sugar',
302     'Etanol',
303     'Viable_yeast',
304     'Dead_yeast',
305     'Neast-assimilable_Nitrogen',
306     'Disolved_CO2',
307     'T',
308     't'
309 ]
310
311 plt.figure(figsize=[15,8])
312 plt.title('Observations_for_' + conds)
313
314 for i in range(obs_exp.shape[1]):
315
316     plt.subplot(2,4,i+1)
317
318     plt.plot(
319         obs_exp[:,i],
320         label='Expert'
321     )
322
323     plt.plot(
324         obs_dag[:,i],
325         '--',
326         label='DAGGER'
327     )
328
329     plt.legend()
330     plt.grid(True)
331
332 plt.tight_layout()
333
334 plt.savefig(
335     os.path.join(
336         path,
337         'obs_comparison.png'
338     )

```

```

339     )
340
341
342     ## ACCIONES
343     act_names = [
344         'T_setpoint',
345         'Nitrogen'
346     ]
347
348     plt.figure(figsize=[15,8])
349     plt.title('Actions_for_' + conds)
350
351     for i in range(act_exp.shape[1]):
352
353         plt.subplot(2,1,i+1)
354
355         plt.plot(
356             act_exp[:,i],
357             label='Expert'
358         )
359
360         plt.plot(
361             act_dag[:,i],
362             '--',
363             label='DAGGER'
364         )
365
366         plt.legend()
367         plt.grid(True)
368
369     plt.tight_layout()
370
371     plt.savefig(
372         os.path.join(
373             path,
374             'act_comparison.png'
375         )
376     )

```

fit_reward.py

```

1  import pickle
2  import numpy as np
3  import torch
4  import torch.nn as nn
5  import torch.optim as optim
6  import matplotlib.pyplot as plt
7
8
9  device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
10
11
12  class RewardNet(nn.Module):
13      def __init__(self, input_dim, output_dim, hidden: int = 256):
14          super().__init__()
15
16          self.net = nn.Sequential(

```

```

17         nn.Linear(input_dim, hidden), nn.ReLU(),
18         nn.Linear(hidden, hidden), nn.ReLU(),
19         nn.Dropout(0.3),
20         nn.Linear(hidden, output_dim))
21
22     def forward(self, obs: torch.Tensor) -> torch.Tensor:
23         x = self.net(obs)
24         return x
25
26     def get_difference_logit(self, p_A, p_B):
27         return self.forward(p_A) - self.forward(p_B)
28
29
30
31
32 # divisi n de dataset 70 train 15 val 15 test
33 def split_dataset(dataset):
34     N = len(dataset)
35
36     idx = np.random.permutation(N)
37
38     n_train = int(0.7*N)
39     n_val = int(0.15*N)
40
41     train_idx = idx[:n_train]
42     val_idx = idx[n_train:n_train+n_val]
43     test_idx = idx[n_train+n_val:]
44
45     return {"train": train_idx, "val": val_idx, "test": test_idx}
46
47
48 ### PERFILES
49 FEATURES = [
50     "anthocyanin",
51     "tannin",
52     "catechin",
53     "color_intensity",
54     "glycerol",
55     "volatile_acidity",
56     "ethanol_pct",
57     "residual_sugar",
58     "fermentation_h"
59 ]
60
61 def profile_to_vec(profile):
62     return np.array(
63         [profile[k] for k in FEATURES], dtype=np.float32
64     )
65
66
67 asd = input("Entrenar modelo o cargar modelo? ([1] train / [2] load): ")
68 modo = "train" if asd == "1" else "load"
69
70 ##### CARGAR DATASET
71 dataset = pickle.load(open(r"p2\data\comparisons.pkl", "rb"))
72
73 X_A = []
74 X_B = []
75 Y = []

```

```

76
77 for sample in dataset:
78     X_A.append(profile_to_vec(
79         sample["profile_A"]
80     )
81 )
82
83     X_B.append(profile_to_vec(
84         sample["profile_B"]
85     )
86 )
87
88     Y.append(1 if sample["winner"] == "A" else 0)
89
90 X_A = np.array(X_A)
91 X_B = np.array(X_B)
92 Y = np.array(Y)
93
94 ## splitteamos ahora a train, val y test
95 dataset_splits = split_dataset(dataset)
96
97 train_idx = dataset_splits["train"]
98 val_idx = dataset_splits["val"]
99 test_idx = dataset_splits["test"]
100
101 all_profiles = np.concatenate(
102     [X_A[train_idx], X_B[train_idx]],
103     axis=0
104 )
105
106 ## NORMALIZAR
107 mu = all_profiles.mean(axis=0)
108 std = all_profiles.std(axis=0) + 1e-8
109
110 X_A = (X_A - mu) / std
111 X_B = (X_B - mu) / std
112
113 ## CONJUNTOS train/val/test
114 XA_train = torch.tensor(X_A[train_idx], dtype=torch.float32)
115 XB_train = torch.tensor(X_B[train_idx], dtype=torch.float32)
116 Y_train = torch.tensor(Y[train_idx], dtype=torch.float32)
117
118 XA_val = torch.tensor(X_A[val_idx], dtype=torch.float32)
119 XB_val = torch.tensor(X_B[val_idx], dtype=torch.float32)
120 Y_val = torch.tensor(Y[val_idx], dtype=torch.float32)
121
122 XA_test = torch.tensor(X_A[test_idx], dtype=torch.float32)
123 XB_test = torch.tensor(X_B[test_idx], dtype=torch.float32)
124 Y_test = torch.tensor(Y[test_idx], dtype=torch.float32)
125
126 ### crear model y entrenar
127 model = RewardNet(
128     input_dim=len(FEATURES),
129     output_dim=1
130 ).to(device)
131
132 criterion = nn.BCEWithLogitsLoss()
133
134 optimizer = optim.Adam(model.parameters(), lr=1e-3)

```

```

135
136
137 ### loop
138 epochs = 100
139
140 train_losses = []
141 val_losses = []
142
143 best_val_loss = np.inf
144
145 if modo == "train":
146     for epoch in range(epochs):
147         if epoch % 10 == 0:
148             print(f"Epoch_{epoch}/_{epochs} Train_loss:_{train_losses[-1]}_if_
149                 train_losses_else_'N/A'} Val_loss:_{val_losses[-1]}_if_val_losses_
150                 else_'N/A'")
151
152             model.train()
153
154             logits = model.get_difference_logit(
155                 XA_train.to(device),
156                 XB_train.to(device)
157             ).squeeze()
158
159             loss = criterion(
160                 logits,
161                 Y_train.to(device)
162             )
163
164             optimizer.zero_grad()
165             loss.backward()
166             optimizer.step()
167
168             train_losses.append(loss.item())
169
170             # eval
171             model.eval()
172
173             with torch.no_grad():
174                 val_logits = model.get_difference_logit(
175                     XA_val.to(device),
176                     XB_val.to(device)
177                 ).squeeze()
178
179                 val_loss = criterion(
180                     val_logits,
181                     Y_val.to(device)
182                 )
183
184                 val_losses.append(val_loss.item())
185
186                 if val_loss.item() < best_val_loss:
187                     best_val_loss = val_loss.item()
188                     torch.save(model.state_dict(), "best_reward_model.pt")
189
190             ### cargar mejor modelo
191             model.load_state_dict(
192                 torch.load("best_reward_model.pt", map_location=device)
193             )

```

```

192
193 ## accuracy
194 model.eval()
195
196 with torch.no_grad():
197     logits = model.get_difference_logit(
198         XA_test.to(device),
199         XB_test.to(device)
200     ).squeeze()
201
202     preds = (logits > 0).cpu().numpy()
203
204     acc = np.mean(preds == Y_test.numpy())
205
206     print(f"Test accuracy: {acc:.4f}")
207
208 ### MATRIZ DE CONFUSI N
209 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, r2_score
210 from scripts.quality_functions import get_wine_quality_score
211
212 cm = confusion_matrix(Y_test.numpy(), preds)
213 disp = ConfusionMatrixDisplay(confusion_matrix=cm)
214
215 disp.plot()
216 plt.savefig("results/p2/confusion_matrix.png", dpi=300)
217
218 # R^2
219 unique_profiles = []
220 seen = set()
221
222 for sample in dataset:
223     for profile in (sample["profile_A"], sample["profile_B"]):
224         key = tuple(profile[k] for k in FEATURES)
225         if key not in seen:
226             seen.add(key)
227             unique_profiles.append(profile)
228
229 true_scores = []
230 pred_scores = []
231
232 model.eval()
233 with torch.no_grad():
234     for profile in unique_profiles:
235         x = profile_to_vec(profile)
236         x = (x - mu) / std
237         x = torch.tensor(x, dtype=torch.float32, device=device).unsqueeze(0)
238
239         true_scores.append(get_wine_quality_score(profile))
240         pred_scores.append(model(x).squeeze().item())
241
242
243 pred_arr = np.array(pred_scores)
244 true_arr = np.array(true_scores)
245
246 # ajuste a escala de 0 a 8
247 true_min, true_max = true_arr.min(), true_arr.max()
248 pred_rescaled = (pred_arr - pred_arr.min()) / (pred_arr.max() - pred_arr.min())
249 pred_rescaled = pred_rescaled * (true_max - true_min) + true_min
250

```



```

251 r2 = r2_score(true_arr, pred_rescaled)
252 print(f"R^2: {r2:.4f}")
253
254 plt.figure(figsize=(6, 6))
255 plt.scatter(true_arr, pred_rescaled, alpha=0.6) # pred_rescaled
256 lims = [
257     min(min(true_arr), min(pred_rescaled)),
258     max(max(true_arr), max(pred_rescaled)),
259 ]
260
261 plt.plot(lims, lims, "r--", linewidth=1)
262 plt.xlabel("true_scores")
263 plt.ylabel("pred_scores")
264 plt.title("True vs Predicted reward")
265 plt.grid(True, alpha=0.3)
266 plt.tight_layout()
267 plt.savefig("results/p2/reward_fit.png", dpi=300)
268 plt.show()

```

p2d.py

```

1  import os, copy, pickle
2  import sys
3  import numpy as np
4  import torch
5  import torch.nn.functional as F
6  import matplotlib.pyplot as plt
7  from dataclasses import dataclass
8  from typing import Optional
9  import torch.nn as nn
10
11  ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
12  sys.path.insert(0, ROOT)
13
14  # imports locales
15  from p3.TD3 import Actor, CriticTwin, ReplayBuffer, rolling_mean
16  from fit_reward import RewardNet, FEATURES, profile_to_vec
17  from dagger import DAGGER
18
19  from scripts.fermentation_env import WineFermentationEnv
20  from scripts.expert_controller import WineExpertController
21  from scripts.quality_functions import maceration_extraction, get_wine_quality_score
22
23  # config
24  device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
25  OUT = "results/p2/d"
26  os.makedirs(OUT, exist_ok=True)
27
28  N_EPISODES = 400
29  NOISE_START = 0.15
30  NOISE_END = 0.01
31
32  @dataclass
33  class TD3Config:
34      gamma: float = 0.99
35      tau: float = 0.003
36      policy_delay: int = 2

```

```

37
38 actor_lr: float = 3e-5
39 critic_lr: float = 3e-4
40
41 batch_size: int = 128
42 replay_size: int = 500_000
43
44 start_steps: int = 10_000
45 update_after: int = 8_000
46 update_every: int = 1
47
48 grad_clip_norm: Optional[float] = 5
49
50 policy_noise_frac: float = 0.05
51 noise_clip_frac: float = 0.10
52 exploration_start_frac: float = 0.15
53 exploration_end_frac: float = 0.02
54 episode_horizon_steps: Optional[int] = None
55 temp_slew_rate: float = 0.5
56 exploration_noise_abs: Optional[np.ndarray] = None
57
58 ### TD3 WINE
59 class WineTD3:
60     def __init__(self, obs_dim, act_dim, act_low, act_high,
61                 reward_net: RewardNet, reward_mu, reward_std, dagger: DAGGER, config:
62                     TD3Config):
63
64         self.config = config
65         self.cfg = config
66         self.dagger = dagger
67         self.reward_net = reward_net
68         self.reward_mu = reward_mu
69         self.reward_std = reward_std
70         self.device = device
71         self.max_episode_steps = config.episode_horizon_steps
72
73         lo, hi = np.asarray(act_low, dtype=np.float32), np.asarray(act_high, dtype
74             =np.float32)
75         self.act_low_np = lo
76         self.act_high_np = hi
77         self.act_low_t = torch.as_tensor(lo, device=device)
78         self.act_high_t = torch.as_tensor(hi, device=device)
79         self.config.exploration_noise_abs = np.array([0.05, 0.30], dtype=np.float32) if
80             self.config.exploration_noise_abs is None else np.asarray(self.config.
81                 exploration_noise_abs, dtype=np.float32)
82         self.act_low = self.act_low_t
83         self.act_high = self.act_high_t
84
85         self.actor = Actor(obs_dim, act_dim, lo, hi).to(device)
86         self.critic = CriticTwin(obs_dim, act_dim).to(device)
87         self.actor_targ = copy.deepcopy(self.actor)
88         self.critic_targ = copy.deepcopy(self.critic)
89         for p in list(self.actor_targ.parameters()) + list(self.critic_targ.parameters()
90             ):
91             p.requires_grad = False
92
93         self.actor_opt = torch.optim.Adam(self.actor.parameters(), lr=config.actor_lr)
94         self.critic_opt = torch.optim.Adam(self.critic.parameters(), lr=config.critic_lr
95             )

```

```

90         self.total_it = 0
91
92     # init from dagger
93     def init_from_dagger(self, sd):
94         self.actor.load_state_dict(sd)
95         self.actor_targ.load_state_dict(sd)
96         print("Actor initialized from DAGGER.")
97
98     # normalizaci n de estados
99     def _norm(self, obs):
100         mn, mx = self.dagger.state_lims
101         return (obs - mn) / (mx - mn + 1e-8).astype(np.float32)
102
103     # reward (SPARSE)
104     def _reward(self, env, done):
105
106         # damos reward 0 si no termin
107         if not done:
108             return 0.0
109
110         profile = maceration_extraction(env.get_trajectory())
111         x = (profile_to_vec(profile) - self.reward_mu) / self.reward_std
112
113         ## reward por perfil de vinos!
114         with torch.no_grad():
115             r = self.reward_net(torch.tensor(x, device=device).unsqueeze(0)).item()
116
117         finished = env._state.S < 1.0 ### premio para gu ar si es que termina
118             fermentaci n! (para gu ar exploraci n)
119         completion_bonus = 2.0 if finished else 0.0
120
121         return float(r) + completion_bonus
122
123     def _episode_terminal(self, env_done: bool, step_idx: int) -> bool:
124         if self.max_episode_steps is None:
125             return env_done
126         return env_done or (step_idx + 1 >= self.max_episode_steps)
127
128     @torch.no_grad()
129     def act(self, obs: np.ndarray, noise_scale=0.0, prev_action=None) -> np.ndarray:
130         obs_t = torch.as_tensor(obs, dtype=torch.float32, device=self.device).unsqueeze(
131             0)
132         a = self.actor(obs_t).cpu().numpy()[0]
133
134         if np.any(np.asarray(noise_scale) > 0.0):
135             noise_abs = self.config.exploration_noise_abs * (noise_scale / max(
136                 NOISE_START, 1e-8))
137             a = a + noise_abs * np.random.randn(*a.shape)
138
139         if prev_action is not None:
140             temp_delta = np.clip(
141                 a[0] - prev_action[0],
142                 -self.config.temp_slew_rate,
143                 self.config.temp_slew_rate,
144             )
145             a[0] = prev_action[0] + temp_delta
146         return np.clip(a, self.act_low_np, self.act_high_np)

```

```

146 def update(self, rb: ReplayBuffer):
147     config = self.config
148     self.total_it += 1
149
150     obs, act, rew, next_obs, done = rb.sample(config.batch_size, self.device)
151     with torch.no_grad():
152         action_range = self.act_high - self.act_low
153
154         noise = torch.randn_like(act) * (config.policy_noise_frac * action_range)
155         noise = noise.clamp(-config.noise_clip_frac * action_range, config.
156                             noise_clip_frac * action_range)
157
158         next_act = torch.clamp((self.actor_targ(next_obs) - self.act_low) /
159                               action_range + noise, min=0, max=1)
160
161         q1_t, q2_t = self.critic_targ(next_obs, next_act)
162         q_t = torch.min(q1_t, q2_t)
163         y = rew + config.gamma * (1.0 - done) * q_t
164         y = torch.clamp(y, -20.0, 10.0)
165
166         q1, q2 = self.critic(obs, (act - self.act_low) / action_range)
167         critic_loss = F.mse_loss(q1, y) + F.mse_loss(q2, y)
168
169         self.critic_opt.zero_grad(set_to_none=True)
170         critic_loss.backward()
171         if config.grad_clip_norm is not None:
172             nn.utils.clip_grad_norm_(self.critic.parameters(), config.grad_clip_norm)
173         self.critic_opt.step()
174
175     info = {
176         "critic_loss": float(critic_loss.item()),
177         "actor_loss": float("nan"),
178         "q1_mean": float(q1.mean().item()),
179         "q1_std": float(q1.std().item()),
180         "target_mean": float(y.mean().item()),
181         "target_std": float(y.std().item()),
182     }
183
184     if self.total_it % config.policy_delay == 0:
185         actor_loss = -self.critic.q1_only(obs, (self.actor(obs) - self.act_low) /
186                                                action_range).mean()
187
188         self.actor_opt.zero_grad(set_to_none=True)
189         actor_loss.backward()
190         if config.grad_clip_norm is not None:
191             nn.utils.clip_grad_norm_(self.actor.parameters(), config.grad_clip_norm)
192         self.actor_opt.step()
193
194         with torch.no_grad():
195             for p, p_t in zip(self.actor.parameters(), self.actor_targ.parameters()):
196                 p_t.data.mul_(1.0 - config.tau).add_(config.tau * p.data)
197             for p, p_t in zip(self.critic.parameters(), self.critic_targ.parameters()):
198                 p_t.data.mul_(1.0 - config.tau).add_(config.tau * p.data)
199
200         info["actor_loss"] = float(actor_loss.item())
201
202     return info

```

```

200
201
202 def train(self, n_episodes=N_EPISODES, noise_start=NOISE_START, noise_end=NOISE_END)
    :
203     obs_dim = len(self.dagger.state_lims[0])
204     rb = ReplayBuffer(obs_dim, len(self.act_low_np), self.config.replay_size)
205
206     ep_returns, true_scores, c_losses, a_losses = [], [], [], []
207     best_score = -np.inf
208     best_path = f"{OUT}/best_td3_wine_policy.pt"
209     warmup_episodes = max(1, int(self.config.exploration_start_frac * n_episodes))
210     decay_episodes = max(warmup_episodes + 1, int((1.0 - self.config.
        exploration_end_frac) * n_episodes))
211
212     for ep in range(1, n_episodes+1):
213         if ep <= warmup_episodes:
214             noise = noise_start
215         elif ep >= decay_episodes:
216             noise = noise_end
217         else:
218             frac = (ep - warmup_episodes) / max(1, decay_episodes - warmup_episodes)
219             noise = noise_start + frac * (noise_end - noise_start)
220         env = WineFermentationEnv(); obs = env.reset(); done = False; ep_r = 0.0
221         a = None
222         step_idx = 0
223
224         while not done:
225             on = self._norm(obs); a = self.act(on, noise, a)
226             nobs, env_done, _ = env.step(a)
227             done = self._episode_terminal(env_done, step_idx)
228             r = self._reward(env, done); ep_r += r
229             rb.add(on, a, r, self._norm(nobs), float(done))
230             obs = nobs
231             if rb.size >= self.config.update_after:
232                 info = self.update(rb)
233                 c_losses.append(info["critic_loss"])
234                 if not np.isnan(info["actor_loss"]):
235                     a_losses.append(info["actor_loss"])
236                 step_idx += 1
237
238             ep_returns.append(ep_r)
239             ts = get_wine_quality_score(maceration_extraction(env.get_trajectory()))
240             true_scores.append(ts)
241             if ts > best_score:
242                 best_score = ts
243                 torch.save(self.actor.state_dict(), best_path)
244             if ep % 20 == 0: print(f"ep={ep}/{n_episodes} reward={ep_r:.3f} score={ts
                :.4f} noise={noise:.3f}")
245
246     return ep_returns, true_scores, c_losses, a_losses
247
248
249 def rollout(self):
250     env = WineFermentationEnv(); obs = env.reset(); done = False
251     states, actions = [], []
252     a = None
253     step_idx = 0
254     while not done:
255         a = self.act(self._norm(obs), prev_action=a)

```

```

256         states.append(obs.copy()); actions.append(a.copy())
257         obs, env_done, _ = env.step(a)
258         done = self._episode_terminal(env_done, step_idx)
259         step_idx += 1
260         profile = maceration_extraction(env.get_trajectory())
261         score = get_wine_quality_score(profile)
262         x = (profile_to_vec(profile)-self.reward_mu)/self.reward_std
263         with torch.no_grad():
264             pred = self.reward_net(torch.tensor(x, device=device).unsqueeze(0)).item()
265         states, actions = _pad_to_horizon(np.array(states), np.array(actions), self.
            max_episode_steps)
266         return states, actions, score, pred
267
268     ### ROLLOUTS DE CADA CONTROLADOR
269     def _pad_to_horizon(states, actions, max_steps):
270         if max_steps is None or len(states) >= max_steps:
271             return states, actions
272
273         pad_n = max_steps - len(states)
274         last_state = states[-1:]
275         last_action = actions[-1:]
276         states = np.concatenate([states, np.repeat(last_state, pad_n, axis=0)], axis=0)
277         actions = np.concatenate([actions, np.repeat(last_action, pad_n, axis=0)], axis=0)
278         return states, actions
279
280
281     def _run_env(actor_fn, max_steps=None):
282         env = WineFermentationEnv()
283         obs = env.reset()
284         done = False
285         states, actions = [], []
286         step_idx = 0
287
288         while not done:
289             a = actor_fn(obs)
290             states.append(obs.copy())
291             actions.append(a.copy())
292             obs, env_done, _ = env.step(a)
293             done = env_done or (max_steps is not None and step_idx + 1 >= max_steps)
294             step_idx += 1
295
296         profile = maceration_extraction(env.get_trajectory())
297         states, actions = _pad_to_horizon(np.array(states), np.array(actions), max_steps)
298
299         return states, actions, get_wine_quality_score(profile)
300
301     def expert_rollout(max_steps=None):
302         controller = WineExpertController()
303         controller.reset()
304
305         return _run_env(lambda obs: controller(obs, float(obs[7])), max_steps=max_steps)
306
307     def dagger_rollout(actor, dagger, max_steps=None):
308         def act(obs):
309             mn, mx = dagger.state_lims
310             on = ((obs-mn) / (mx-mn+1e-8)).astype(np.float32)
311             with torch.no_grad():
312                 return actor(torch.tensor(on, device=device).unsqueeze(0)).cpu().numpy()[0]
313         return _run_env(act, max_steps=max_steps)

```

```

314
315 ##### PLOTS!
316 OBS_NAMES = ["Res. Sugar", "Etanol", "Yeast vivo", "Yeast muerto", "N asim.", "C O dis.", "T",
317              ", "t[h]"]
318 ACT_NAMES = ["T setpoint", "N a adido"]
319
320 def plot_training(ep_returns, true_scores, c_losses, a_losses):
321     fig, axes = plt.subplots(2, 2, figsize=(14, 8))
322     fig.suptitle("Entrenamiento TD3 Sparse Reward")
323     for ax, data, title, color in zip(axes.flatten(),
324                                     [ep_returns, true_scores, c_losses, a_losses],
325                                     ["Retorno por poca ", "Score verdadero", "Critic Loss", "Actor Loss"],
326                                     ["steelblue", "seagreen", "tomato", "darkorange"]):
327         ax.plot(data, alpha=0.35, color=color)
328         ax.plot(rolling_mean(data, 20), color=color, linewidth=2)
329         ax.set_title(title); ax.grid(True, alpha=0.3)
330     plt.tight_layout()
331     plt.savefig(f"{OUT}/training_curves.png", dpi=150); plt.show()
332
333 def plot_comparison(rl_runs, dag_runs, exp_runs):
334     s_rl = np.stack([r[0] for r in rl_runs], axis=0)
335     a_rl = np.stack([r[1] for r in rl_runs], axis=0)
336     sc_rl = np.array([r[2] for r in rl_runs], dtype=np.float32)
337
338     s_dag = np.stack([r[0] for r in dag_runs], axis=0)
339     a_dag = np.stack([r[1] for r in dag_runs], axis=0)
340     sc_dag = np.array([r[2] for r in dag_runs], dtype=np.float32)
341
342     s_exp = np.stack([r[0] for r in exp_runs], axis=0)
343     a_exp = np.stack([r[1] for r in exp_runs], axis=0)
344     sc_exp = np.array([r[2] for r in exp_runs], dtype=np.float32)
345
346     s_rl_mean, s_rl_std = s_rl.mean(axis=0), s_rl.std(axis=0)
347     s_dag_mean, s_dag_std = s_dag.mean(axis=0), s_dag.std(axis=0)
348     s_exp_mean, s_exp_std = s_exp.mean(axis=0), s_exp.std(axis=0)
349
350     a_rl_mean, a_rl_std = a_rl.mean(axis=0), a_rl.std(axis=0)
351     a_dag_mean, a_dag_std = a_dag.mean(axis=0), a_dag.std(axis=0)
352     a_exp_mean, a_exp_std = a_exp.mean(axis=0), a_exp.std(axis=0)
353
354     fig, axes = plt.subplots(3, 3, figsize=(16, 11))
355     fig.suptitle("Comparación: RL vs DAGGER vs Experto")
356     for i, ax in enumerate(axes.flatten()):
357         if i < s_rl_mean.shape[1]:
358             ax.plot(s_exp_mean[:, i], "k-", lw=1.2, label="Experto")
359             ax.fill_between(np.arange(s_exp_mean.shape[0]), s_exp_mean[:, i] - s_exp_std[:, i], s_exp_mean[:, i] + s_exp_std[:, i], color="k", alpha=0.12)
360             ax.plot(s_dag_mean[:, i], "b--", lw=1.2, label="DAGGER")
361             ax.fill_between(np.arange(s_dag_mean.shape[0]), s_dag_mean[:, i] - s_dag_std[:, i], s_dag_mean[:, i] + s_dag_std[:, i], color="b", alpha=0.12)
362             ax.plot(s_rl_mean[:, i], "r-", lw=1.2, label="TD3-RL")
363             ax.fill_between(np.arange(s_rl_mean.shape[0]), s_rl_mean[:, i] - s_rl_std[:, i], s_rl_mean[:, i] + s_rl_std[:, i], color="r", alpha=0.12)
364             ax.set_title(OBS_NAMES[i] if i < len(OBS_NAMES) else f"obs[{i}]")
365             ax.legend(fontsize=7); ax.grid(True, alpha=0.3)
366     plt.tight_layout(); plt.savefig(f"{OUT}/states_comparison.png", dpi=150); plt.show()
367
368     fig, axes = plt.subplots(1, 2, figsize=(12, 4))

```

```

368 fig.suptitle("Acciones: RL vs DAGGER vs Experto")
369 for i, ax in enumerate(axes):
370     ax.plot(a_exp_mean[:,i], "k-", lw=1.2, label="Experto")
371     ax.fill_between(np.arange(a_exp_mean.shape[0]), a_exp_mean[:,i] - a_exp_std[:,i],
372                    a_exp_mean[:,i] + a_exp_std[:,i], color="k", alpha=0.12)
372     ax.plot(a_dag_mean[:,i], "b--", lw=1.2, label="DAGGER")
373     ax.fill_between(np.arange(a_dag_mean.shape[0]), a_dag_mean[:,i] - a_dag_std[:,i],
374                    a_dag_mean[:,i] + a_dag_std[:,i], color="b", alpha=0.12)
374     ax.plot(a_rl_mean[:,i], "r-", lw=1.2, label="TD3-RL")
375     ax.fill_between(np.arange(a_rl_mean.shape[0]), a_rl_mean[:,i] - a_rl_std[:,i],
376                    a_rl_mean[:,i] + a_rl_std[:,i], color="r", alpha=0.12)
376     ax.set_title(ACT_NAMES[i]); ax.legend(); ax.grid(True, alpha=0.3)
377 plt.tight_layout(); plt.savefig(f"{OUT}/actions_comparison.png", dpi=150); plt.
    show()

378
379 fig, ax = plt.subplots(figsize=(6,4))
380 scores_mean = [float(sc_exp.mean()), float(sc_dag.mean()), float(sc_rl.mean())]
381 scores_std = [float(sc_exp.std()), float(sc_dag.std()), float(sc_rl.std())]
382 bars = ax.bar(["Experto", "DAGGER", "TD3-RL"], scores_mean, yerr=scores_std, color=["
    gray", "steelblue", "tomato"], edgecolor="k", alpha=0.85, capsize=5)
383 for b, s in zip(bars, scores_mean): ax.text(b.get_x()+b.get_width()/2, b.get_height
    (+).02, f"{s:.3f}", ha="center")
384 ax.set_ylabel("Score de calidad"); ax.set_title("Scores finales"); ax.grid(True,
    alpha=0.3, axis="y")
385 plt.tight_layout(); plt.savefig(f"{OUT}/score_comparison.png", dpi=150); plt.show
    ()

386
387 #####
388 ##          MAIN          ##
389 #####
390
391 if __name__ == "__main__":
392     # Cargar datos y NORMALIZAR
393     with open("p2/data/expert_trajectories.pkl", "rb") as f: expert_trajs = pickle.load(f
    )
394
395     expert_steps = []
396     for traj in expert_trajs:
397         expert_steps.append(len(traj["trajectory"]))
398     episode_horizon_steps = int(np.max(expert_steps))
399
400     dag_helper = DAGGER()
401     dag_helper.state_lims = dag_helper.get_normalization_params(expert_trajs)
402     obs_dim = len(dag_helper.state_lims[0]); act_dim = 2
403
404     # DAGGER (pretrained)
405     dagger_actor = Actor(obs_dim, act_dim, dag_helper.act_lims[0], dag_helper.act_lims
    [1]).to(device)
406     dagger_actor.load_state_dict(torch.load("results/p2/dagger/dagger_policy.pt",
    map_location=device))
407     dagger_actor.eval()
408
409     # Red de Rewards
410     dataset = pickle.load(open("p2/data/comparisons.pkl", "rb"))
411     all_vecs = np.array([profile_to_vec(s[k]) for s in dataset for k in ("profile_A", "
    profile_B")])
412     mu, std = all_vecs.mean(0), all_vecs.std(0)+1e-8
413
414     rnet = RewardNet(len(FEATURES), 1).to(device)

```



```

415 rnet.load_state_dict(torch.load("best_reward_model.pt", map_location=device))
416 rnet.eval()
417
418 # TD3
419 cfg = TD3Config(episode_horizon_steps=episode_horizon_steps)
420 agent = WineTD3(obs_dim, act_dim, dag_helper.act_lims[0], dag_helper.act_lims[1],
421               rnet, mu, std, dag_helper, cfg)
422 agent.init_from_dagger(dagger_actor.state_dict())
423
424 # TRAIN
425 ep_returns, true_scores, c_losses, a_losses = agent.train(n_episodes=N_EPISODES)
426 agent.actor.load_state_dict(torch.load(f"{OUT}/best_td3_wine_policy.pt",
427                                     map_location=device))
428 agent.actor_targ.load_state_dict(agent.actor.state_dict())
429 torch.save(agent.actor.state_dict(), f"{OUT}/td3_wine_policy.pt")
430
431 # PLOT
432 plot_training(ep_returns, true_scores, c_losses, a_losses)
433
434 rl_res = [agent.rollout() for _ in range(5)]
435 dag_res = [dagger_rollout(dagger_actor, dag_helper, max_steps=episode_horizon_steps)
436            for _ in range(5)]
437 exp_res = [expert_rollout(max_steps=episode_horizon_steps) for _ in range(5)]
438
439 for name, res in [("Experto", exp_res), ("DAGGER", dag_res), ("TD3-RL", rl_res)]:
440     scores = [r[2] for r in res]
441     print(f"{name:<10} score={np.mean(scores):.4f} ± {np.std(scores):.4f}")
442
443 plot_comparison(rl_res, dag_res, exp_res)

```

residual_controller.py

```

1 import copy
2 import os
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import torch
6
7 from scripts.Quadrotor3D import Quadrotor3D
8 from scripts.QuadrotorTrajectories import QuadrotorTrajectory
9
10 from LQR import LQR, transform_references
11 from TD3 import TD3Agent, TD3Config, ReplayBuffer, linear_schedule, set_seed
12
13 ### DOMAIN RANDOMIZATION
14 Ts = 0.01
15 known_params = {
16     "m": 0.027, # kg
17     "L": 0.0397, # m
18     "g": 9.8, # m/s
19 }
20
21 # rangos de randomizaci n por ep
22 unknown_params = {
23     "Ixx": [0.6e-5, 3.0e-5],
24     "Iyy": [0.6e-5, 3.0e-5],

```

```

25     "Izz": [1.0e-5, 5.0e-5],
26     "KF": [2.0e-10, 4.5e-10],
27     "KM": [4.0e-12, 1.3e-11],
28 }
29
30 def sample_dr_params(rng, known, unknown):
31     out = dict(known)
32     # iteramos para cada uno y agregamos ruido
33     for k, (low, high) in unknown.items():
34         out[k] = rng.uniform(low=low, high=high)
35
36     return out
37
38 rng = np.random.default_rng(0)
39
40 ### ENVIRONMENT
41 env_train = Quadrotor3D(Ts=Ts) # con DR
42 env_eval = Quadrotor3D(Ts=Ts) # fijo, para eval
43
44 ### TRAJECTORY
45 traj_gen = QuadrotorTrajectory(sample_time=Ts)
46 refs_raw = traj_gen.helix(length=12, num_cycles=3, radius=1, rise=0.5, center=(0., 0.,
47     0.5))
48 references = transform_references(refs_raw)
49
50 ## LQR BASE (pi_h)
51 lqr_model_params = sample_dr_params(rng, known_params, unknown_params)
52 lqr_model = Quadrotor3D(Ts=Ts, params=lqr_model_params)
53
54 Q_lqr = np.diag([
55     5., 2., # x,
56     5., 2., # y,
57     5., 2., # z,
58     1., 1., 0.1, # , ,
59     0.1, 0.1, 0.1, # p, q, r
60 ])
61 R_lqr = np.eye(4) * 0.01
62
63 f_hover = known_params['m'] * known_params['g'] / 4.0
64 u_min = 0.0
65 u_max = f_hover * 2.5
66
67 lqr = LQR(lqr_model, n=12, m=4, Q=Q_lqr, R=R_lqr, u_lims=[u_min, u_max])
68
69 ### CONTROLADOR RESIDUAL (pi_r) TD3
70 ### reward function residual
71 def reward_function(state, action, next_state, reference):
72     # minimizar error de ref
73     pos = state[[0, 2, 4]] # x, y, z
74     ref_pos = reference[[0, 2, 4]]
75     dist = np.linalg.norm(pos - ref_pos)
76
77     action_penalty = 0.01 * np.linalg.norm(action) # castigando acci n grande
78
79     reward = -dist - action_penalty
80     return reward
81
82 ### Agente

```

```

83 delta_max = 0.5 * f_hover # maximo cambio permitido al LQR
84
85 agent = TD3Agent(
86     reward_function=reward_function,
87     obs_dim=12*2 + 4, # estado actual + referencia + acci n del LQR
88     act_dim=4,
89     act_low=-delta_max * np.ones(4),
90     act_high=delta_max * np.ones(4),
91     cfg=TD3Config(),
92     device=torch.device("cuda" if torch.cuda.is_available() else "cpu"),
93     max_pos_error=2.0,
94     max_angle_deg=60.0,
95 )
96
97
98 def train_residual( agent, lqr, env, references, known_params, unknown_params, rng,
99     steps=250_000, seed=0, eval_every=1_000):
100     set_seed(seed)
101
102     # dims
103     obs_dim = 12 * 2 + 4
104     act_dim = 4
105     max_steps = references.shape[1]
106
107     # init replay buffer
108     rb = ReplayBuffer(obs_dim=obs_dim, act_dim=act_dim, capacity=agent.cfg.replay_size)
109
110     # random params de cada episodio
111     # INIT RANDOM PROCESS N FOR EPXLORATION
112     params = sample_dr_params(rng, known_params, unknown_params)
113     state, _ = env.reset(x0=references[:, 0], params=params)
114
115     t_idx = 0
116     ref = references[:, t_idx]
117     u_lqr = lqr(state, ref)
118     obs = agent.build_obs(state, ref, u_lqr, env.obs_norm, f_hover)
119     ep_return = 0.0
120     ep_len = 0
121     ep_num = 0
122     best_return = -np.inf
123     best_actor_state = None
124     ep_length_history = []
125     window_info = []
126     metrics = {
127         "episode_returns": [],
128         "update_steps": [],
129         "critic_loss": [],
130         "actor_loss": [],
131         "q1_mean": [],
132         "q1_std": [],
133         "target_mean": [],
134         "target_std": [],
135     }
136
137     for t in range(1, steps + 1):
138         # GET u_t = pi_theta(s_t) + noise(t)
139         noise_frac = linear_schedule(
140             t,
141             steps,

```

```

141         agent.cfg.exploration_start_frac,
142         agent.cfg.exploration_end_frac,
143     )
144     noise_scale = noise_frac * (agent.act_high_np - agent.act_low_np)
145
146     if t < agent.cfg.start_steps:
147         delta_u = np.random.uniform(agent.act_low_np, agent.act_high_np).astype(np.
148             float32)
149     else:
150         delta_u = agent.act(obs, noise_scale=noise_scale)
151
152     # GET u'_t = u_t + pi_h(s_t) (con pi_h == LQR)
153     u_total = np.clip(u_lqr + delta_u, u_min, u_max)
154
155     # GET s_{t+1} ~ p(- | s_t, u'_t)
156     next_state, _ = env.step(u_total)
157
158     t_idx = min(env.t, max_steps - 1)
159     next_ref = references[:, t_idx]
160
161     reward = reward_function(state, u_total, next_state, next_ref)
162
163     traj_done = env.t >= max_steps
164     unstable = agent.is_unstable(next_state, next_ref)
165     if unstable:
166         reward -= 10.0 * (max_steps - env.t) # penalty sobre fun reward base para
167         castigar inestabilidad
168
169     done = traj_done or unstable
170     next_u_lqr = lqr(next_state, next_ref)
171     next_obs = agent.build_obs(next_state, next_ref, next_u_lqr, env.obs_norm,
172         f_hover)
173
174     ## STORE (s_t, u_t, s_{t+1}, r_t) INTO REPLAY BUFFER
175     rb.add(obs, delta_u, reward, next_obs, float(done))
176
177     state = next_state
178     ref = next_ref
179     u_lqr = next_u_lqr
180     obs = next_obs
181
182     ep_return += float(reward)
183     ep_len += 1
184
185     if t >= agent.cfg.update_after and t % agent.cfg.update_every == 0 and rb.size
186         >= agent.cfg.batch_size:
187         info = agent.update(rb)
188         window_info.append(info)
189
190     if done: # ep terminado
191         ep_num += 1
192         print(
193             f"[residual-train] step={t:>7d} ep={ep_num:>4d} "
194             f"return={ep_return:>9.2f} len={ep_len:>4d}"
195         )
196
197     # guardamos model si es mejor q el q tenemos
198     if ep_return > best_return:
199         best_return = ep_return

```

```

196         best_actor_state = {
197             k: v.detach().cpu().clone() for k, v in agent.actor.state_dict().
                items()
198         }
199
200     # nueva randomizaci n para el pr ximo episodio
201     params = sample_dr_params(rng, known_params, unknown_params)
202     state, _ = env.reset(x0=references[:, 0], params=params)
203     t_idx = 0
204     ref = references[:, t_idx]
205     u_lqr = lqr(state, ref)
206     obs = agent.build_obs(state, ref, u_lqr, env.obs_norm, f_hover)
207
208     metrics["episode_returns"].append(ep_return)
209     ep_length_history.append(ep_len)
210     ep_return = 0.0
211     ep_len = 0
212
213     if t % eval_every == 0:
214
215         if window_info:
216             cl = float(np.mean([i["critic_loss"] for i in window_info]))
217             al_v = [i["actor_loss"] for i in window_info if not np.isnan(i["
                actor_loss"])]
218             al = float(np.mean(al_v)) if al_v else float("nan")
219             qm = float(np.mean([i["q1_mean"] for i in window_info]))
220             qs = float(np.mean([i["q1_std"] for i in window_info]))
221             tm = float(np.mean([i["target_mean"] for i in window_info]))
222             ts = float(np.mean([i["target_std"] for i in window_info]))
223
224             metrics["update_steps"].append(t)
225             metrics["critic_loss"].append(cl)
226             metrics["actor_loss"].append(al)
227             metrics["q1_mean"].append(qm)
228             metrics["q1_std"].append(qs)
229             metrics["target_mean"].append(tm)
230             metrics["target_std"].append(ts)
231
232     results_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'results
        ', 'p3'))
233     os.makedirs(results_dir, exist_ok=True)
234     best_model_path = os.path.join(results_dir, "best_td3_residual_actor.pt")
235     if best_actor_state is not None:
236         torch.save(best_actor_state, best_model_path)
237         print(f"[residual-train] no saved to {best_model_path} | best_return={
            best_return:.2f}")
238     else:
239         print("[residual-train] no completed episode with improvement; existing
            checkpoint was kept.")
240
241
242     plt.figure(figsize=(15, 8))
243     plt.subplot(2, 1, 1)
244     plt.plot(ep_length_history, label='Len')
245     plt.grid(True)
246     plt.subplot(2, 1, 2)
247     plt.plot(metrics['episode_returns'], label='Return')
248     plt.grid(True)
249     plt.legend()

```

```

250     plt.tight_layout()
251     plt.savefig(os.path.join(results_dir, 'td3_residual_train_episode_returns.png'), dpi
                =200, bbox_inches='tight')
252     plt.show()
253
254     return metrics
255
256
257 def evaluate_residual(agent, lqr, env, references, known_params, f_hover, u_min, u_max):
258     results_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'results
                ', 'p3'))
259     os.makedirs(results_dir, exist_ok=True)
260
261     agent.actor.eval()
262     act_dim = 4
263     max_steps = references.shape[1]
264
265     state, _ = env.reset(x0=references[:, 0])
266     env_lqr = copy.deepcopy(env)
267     state_lqr, _ = env_lqr.reset(x0=references[:, 0])
268     t_idx = 0
269
270     states = []
271     states_lqr = []
272     refs = []
273     u_totals = []
274     rewards = []
275     pos_errors = []
276     unstable = False
277
278     for k in range(max_steps - 1):
279         ref = references[:, t_idx]
280         u_lqr = lqr(state, ref)
281         obs = agent.build_obs(state, ref, u_lqr, env.obs_norm, f_hover)
282         delta_u = agent.act(obs, noise_scale=0.0)
283         u_total = np.clip(u_lqr + delta_u, u_min, u_max)
284
285         u_lqr_only = lqr(state_lqr, ref)
286
287         next_state, _ = env.step(u_total)
288         next_state_lqr, _ = env_lqr.step(u_lqr_only)
289         t_idx = min(env.t, max_steps - 1)
290         next_ref = references[:, t_idx]
291
292         reward = reward_function(state, u_total, next_state, next_ref)
293         traj_done = env.t >= max_steps
294         unstable = agent.is_unstable(next_state, next_ref)
295         if unstable:
296             reward -= 10.0 * (max_steps - env.t)
297
298         states.append(np.asarray(next_state))
299         states_lqr.append(np.asarray(next_state_lqr))
300         refs.append(np.asarray(next_ref))
301         u_totals.append(np.asarray(u_total))
302         rewards.append(float(reward))
303         pos_errors.append(float(np.linalg.norm(next_state[[0, 2, 4]] - next_ref[[0, 2,
                4]])))
304
305         state = next_state

```

```

306         state_lqr = next_state_lqr
307         if traj_done or unstable:
308             break
309
310     states = np.asarray(states)
311     states_lqr = np.asarray(states_lqr)
312     refs = np.asarray(refs)
313     actions = np.asarray(u_totals)
314     rewards = np.asarray(rewards)
315     pos_errors = np.asarray(pos_errors)
316
317     total_return = float(np.sum(rewards)) if len(rewards) > 0 else 0.0
318     mean_reward = float(np.mean(rewards)) if len(rewards) > 0 else 0.0
319     rmse_pos = float(np.sqrt(np.mean(pos_errors ** 2))) if len(pos_errors) > 0 else np.
        nan
320
321     t = np.arange(len(states)) * env.Ts
322
323
324     plt.figure(figsize=(15, 7))
325
326     plt.subplot(3, 1, 1)
327     plt.plot(t, refs[:, 0], "k--", label="x_ref")
328     plt.plot(t, states_lqr[:, 0], label="x_LQR")
329     plt.plot(t, states[:, 0], label="x_Residual")
330     plt.ylabel("x[m]")
331     plt.grid(True)
332     plt.legend()
333
334     plt.subplot(3, 1, 2)
335     plt.plot(t, refs[:, 2], "k--", label="y_ref")
336     plt.plot(t, states_lqr[:, 2], label="y_LQR")
337     plt.plot(t, states[:, 2], label="y_Residual")
338     plt.ylabel("y[m]")
339     plt.grid(True)
340     plt.legend()
341
342     plt.subplot(3, 1, 3)
343     plt.plot(t, refs[:, 4], "k--", label="z_ref")
344     plt.plot(t, states_lqr[:, 4], label="z_LQR")
345     plt.plot(t, states[:, 4], label="z_Residual")
346     plt.ylabel("z[m]")
347     plt.xlabel("t[s]")
348     plt.grid(True)
349     plt.legend()
350
351     plt.suptitle(
352         f"Residual_eval    return={total_return:.2f}, "
353         f"RMSE_pos={rmse_pos:.3f}, steps={len(states)}, unstable={unstable}"
354     )
355     plt.tight_layout()
356     plt.savefig(os.path.join(results_dir, 'td3_residual_eval_xyz.png'), dpi=200,
        bbox_inches='tight')
357
358     fig = plt.figure(figsize=(10, 7))
359     ax = fig.add_subplot(111, projection="3d")
360
361     ax.plot(
362         refs[:, 0],

```

```

363         refs[:, 2],
364         refs[:, 4],
365         "k--",
366         linewidth=2,
367         label="Reference",
368     )
369
370     ax.plot(
371         states_lqr[:, 0],
372         states_lqr[:, 2],
373         states_lqr[:, 4],
374         linewidth=1.5,
375         label="LQR_trajectory",
376     )
377
378     ax.plot(
379         states[:, 0],
380         states[:, 2],
381         states[:, 4],
382         linewidth=1.5,
383         label="Residual_trajectory",
384     )
385
386     ax.set_xlabel("X[m]")
387     ax.set_ylabel("Y[m]")
388     ax.set_zlabel("Z[m]")
389     ax.set_title("Quadrotor_Residual_ _Reference_vs_Followed_Trajectory")
390     ax.legend()
391     plt.tight_layout()
392     plt.savefig(os.path.join(results_dir, 'td3_residual_eval_trajectory_3d.png'), dpi
393                 =200, bbox_inches='tight')
394
395     plt.figure(figsize=(15, 8))
396
397     plt.subplot(2, 1, 1)
398     for i in range(act_dim):
399         plt.plot(t, actions[:, i], label=f"u{i+1}")
400     plt.axhline(f_hover, linestyle="--", linewidth=1.0, label="hover")
401     plt.ylabel("final_thrust[N]")
402     plt.grid(True)
403     plt.legend()
404
405     plt.subplot(2, 1, 2)
406     plt.plot(t, rewards, label="reward")
407     plt.xlabel("t[s]")
408     plt.ylabel("reward")
409     plt.grid(True)
410     plt.legend()
411
412     plt.suptitle("Residual_actions_and_rewards")
413     plt.tight_layout()
414     plt.savefig(os.path.join(results_dir, 'td3_residual_eval_actions_rewards.png'), dpi
415                 =200, bbox_inches='tight')
416     plt.show()
417
418     print(
419         f"[residual-eval]_return={total_return:.2f}|_"
420         f"mean_reward={mean_reward:.3f}|_"
421         f"rmse_pos={rmse_pos:.3f}|_"

```



```

420         f"steps={len(states)}\n\n"
421         f"unstable={unstable}\n\n"
422     )
423
424     return {
425         "return": total_return,
426         "mean_reward": mean_reward,
427         "rmse_pos": rmse_pos,
428         "unstable": bool(unstable),
429         "steps": int(len(states)),
430         "states": states,
431         "references": refs,
432         "actions": actions,
433         "rewards": rewards,
434         "pos_errors": pos_errors,
435     }
436
437 def plot_residual_vs_LQR(agent, lqr, env, references, known_params, f_hover, u_min,
438     u_max):
439
440     residual_traj = evaluate_residual(agent, lqr, env, references, known_params, f_hover
441         , u_min, u_max)
442     lqr_traj = lqr.simulate(references)
443
444 if __name__ == "__main__":
445     asd = input("Entrenar [y/n]? ")
446     train = False
447     if asd.lower() == "y":
448         train = True
449
450     if train:
451         train_residual(
452             agent=agent,
453             lqr=lqr,
454             env=copy.deepcopy(env_train),
455             references=references,
456             known_params=known_params,
457             unknown_params=unknown_params,
458             rng=rng,
459             steps=250_000,
460             seed=0,
461         )
462     else:
463         results_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', '
464             results', 'p3'))
465         best_model_path = os.path.join(results_dir, "best_td3_residual_actor.pt")
466         if os.path.exists(best_model_path):
467             actor_state = torch.load(best_model_path, map_location=agent.device)
468             agent.actor.load_state_dict(actor_state)
469             print(f"[main] loaded actor checkpoint: {best_model_path}")
470         else:
471             print("[main] checkpoint not found; evaluating with current in-memory actor.
472                 ")
473
474     ## plot trajs y errores
475     evaluate_residual(
476         agent=agent,

```

```

475         lqr=lqr,
476         env=copy.deepcopy(env_eval),
477         references=references,
478         known_params=known_params,
479         f_hover=f_hover,
480         u_min=u_min,
481         u_max=u_max,
482     )

```

LQR_tuning.py

```

1  import numpy as np
2  import torch
3  from LQR import LQR
4  import copy
5  import os
6  from dataclasses import dataclass
7  from typing import Any, Optional
8  import matplotlib.pyplot as plt
9  import time
10 from scipy.linalg import solve_discrete_are
11
12 from LQR import LQR, reward_function_complex, transform_references,
    reward_function_simple
13 from TD3 import TD3Agent, TD3Config, ReplayBuffer, set_seed, linear_schedule
14
15 from scripts.Quadrotor3D import Quadrotor3D
16 from scripts.QuadrotorTrajectories import QuadrotorTrajectory
17
18
19
20 # matrices base
21 Q_lqr = np.diag([
22     5., 2., # x,
23     5., 2., # y,
24     5., 2., # z,
25     1., 1., 0.1, # , ,
26     0.1, 0.1, 0.1, # p, q, r
27 ])
28 R_lqr = np.eye(4) * 0.01
29
30
31 ## dims de estado y acci n
32 N_Q = 12 # q diag tiene 12 elementos
33 N_R = 4 # r diag tiene 4 elementos
34 act_dim = N_Q + N_R
35
36 # escalamiento de ganancias bounded
37 act_low = np.ones(act_dim)*0.1
38 act_high = np.ones(act_dim)*10.0
39
40 ## observaci n del agente TD3
41 def build_obs_tuning(state, ref, obs):
42     # agente ve error de estado + estado actual (no ve u pq no controla)
43     error = state - ref
44     obs = np.concatenate([
45         state / obs,

```

```

46         error / obs,
47     ])
48     return np.clip(obs, -10.0, 10.0).astype(np.float32)
49
50 obs_dim = 24 # (12 de estado + 12 de error)
51
52 # aplicar acci n del TD3 al LQR
53 def apply_action_to_lqr(lqr, action, Q_prev, R_prev):
54     # accion es vec de 16 elems: escalados de cada valor de Q y R
55     q_scales = action[:N_Q] # primeros 12 para Q
56     r_scales = action[N_Q:] # otros para R
57
58     Q_new = np.diag(np.diag(Q_prev) * q_scales)
59     R_new = np.diag(np.diag(R_prev) * r_scales)
60
61     lqr.recompute_K(Q_new, R_new)
62     return Q_new, R_new
63
64 ### loop de entrenamiento (igual al de TD3 pero cambio de acci n y obs)
65 u_max = (Quadrotor3D().p['m'] * Quadrotor3D().p['g'] / 4.0) * 2.5
66 lqr_base = LQR(model=Quadrotor3D(), n=12, m=4, Q=Q_lqr, R=R_lqr, u_lims=[0, u_max])
67
68
69
70 def train(self, env, lqr_base, references, known_params, unknown_params, seed, steps:
71     int=100_000, eval_every: int=1000):
72     set_seed(seed)
73
74     lqr = copy.deepcopy(lqr_base)
75     prev_action = None
76     prev_u_lqr = None
77
78     results_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'results
79         ', 'p3'))
80     os.makedirs(results_dir, exist_ok=True)
81
82     # Initialization
83     f_hover = known_params['m'] * known_params['g'] / 4.0
84     max_steps = references.shape[1]
85
86     # Agent
87     rb = ReplayBuffer(obs_dim, act_dim, capacity=self.cfg.replay_size)
88
89     params = {**known_params, **self.sample_unknown(unknown_params)}
90     state, _ = env.reset(x0=references[:, 0], params=params)
91
92     t_idx = 0
93     ref = references[:, t_idx]
94
95     # acci n inicial 1
96     action = np.ones(act_dim)
97
98     obs = build_obs_tuning(state, ref, env.obs_norm)
99     obs = torch.from_numpy(obs)
100
101     ep_return = 0.0
102     ep_length = 0

```

```

103     ep_num = 0
104
105     # para guardar el modelo
106     best_return = -np.inf
107     best_actor_state = None
108
109     window_info = []
110     ep_length_history = []
111
112     for t in range(1, steps + 1):
113         noise_frac = linear_schedule(t, int(steps), self.cfg.exploration_start_frac,
114                                     self.cfg.exploration_end_frac)
115         noise_scale = noise_frac * (self.act_high_np - self.act_low_np)
116
117         if t < self.cfg.start_steps:
118             action = np.array([np.random.uniform(self.act_low_np[i], self.act_high_np[i]
119                                                  ]) for i in range(act_dim)])
120         else:
121             action = self.act(obs, noise_scale=noise_scale)
122
123         # Step environment
124         # state, _ = env.step(action)
125
126         # state y obs son del LQR
127         apply_action_to_lqr(lqr, action, Q_lqr, R_lqr)
128         u_lqr = lqr(state, ref)
129         state, _ = env.step(u_lqr)
130
131         t_idx = min(env.t, max_steps - 1)
132         ref = references[:, t_idx]
133
134         # Reward va con acci n del LQR, no del TD3 -> indirecto
135         reward = reward_function_complex(ref, state, u_lqr, prev_u_lqr, f_hover)
136
137         # Termination
138         traj_done = (env.t >= max_steps)
139         unstable = self.is_unstable(state, ref)
140         if unstable:
141             reward -= 10 * (max_steps - env.t)
142         done = traj_done or unstable
143
144         ep_return += float(reward)
145         ep_length += 1
146
147         next_obs = torch.from_numpy(build_obs_tuning(state, ref, env.obs_norm))
148
149         # Add to the replay buffer
150         rb.add(obs, action, reward, next_obs, float(done))
151
152         prev_action = action
153         prev_u_lqr = u_lqr
154         obs = next_obs
155
156         if done:
157             ep_num += 1
158             self.metrics["episode_returns"].append(ep_return)
159             # guardamos el mejor
160             if ep_return > best_return:

```

```

160         best_return = ep_return
161         best_actor_state = {
162             k: v.detach().cpu().clone() for k, v in self.actor.state_dict().
                items()
163         }
164
165         # copia del lqr para resetear cada ep
166         lqr = copy.deepcopy(lqr_base)
167         prev_action = None
168         prev_u_lqr = None
169
170         print(
171             f"[train]_step={t:>7d}_ep={ep_num:>4d}"
172             f"return={ep_return:>8.2f}_len={ep_length:>4d}"
173         )
174         ep_length_history.append(ep_length)
175         ep_return = 0.0
176         ep_length = 0
177
178         # Resample randomized parameters for next episode
179         params = {**known_params, **self.sample_unknown(unknown_params)}
180         state, _ = env.reset(x0=references[:, 0], params=params)
181         t_idx = 0
182         ref = references[:, t_idx]
183         action = np.ones(act_dim)
184         # obs = self.build_obs(state, ref, action, env.obs_norm, f_hover)
185         # obs tambi n cambia
186         obs = build_obs_tuning(state, ref, env.obs_norm)
187
188
189
190         if t >= self.cfg.update_after and t % self.cfg.update_every == 0:
191             info = self.update(rb)
192             window_info.append(info)
193
194         if t % eval_every == 0:
195
196             if window_info:
197                 cl = float(np.mean([i["critic_loss"] for i in window_info]))
198                 al_v = [i["actor_loss"] for i in window_info if not np.isnan(i["
                    actor_loss"])]
199                 al = float(np.mean(al_v)) if al_v else float("nan")
200                 qm = float(np.mean([i["q1_mean"] for i in window_info]))
201                 qs = float(np.mean([i["q1_std"] for i in window_info]))
202                 tm = float(np.mean([i["target_mean"] for i in window_info]))
203                 ts = float(np.mean([i["target_std"] for i in window_info]))
204
205                 self.metrics["update_steps"].append(t)
206                 self.metrics["critic_loss"].append(cl)
207                 self.metrics["actor_loss"].append(al)
208                 self.metrics["q1_mean"].append(qm)
209                 self.metrics["q1_std"].append(qs)
210                 self.metrics["target_mean"].append(tm)
211                 self.metrics["target_std"].append(ts)
212
213         if best_actor_state is None:
214             best_actor_state = {
215                 k: v.detach().cpu().clone() for k, v in self.actor.state_dict().items()
216             }

```

```

217
218 best_model_path = os.path.join(results_dir, "best_td3_actor.pt")
219 torch.save(best_actor_state, best_model_path)
220 print(f"[train]_best_model_saved_to_{best_model_path}|_best_return={best_return:.2f}
    ")
221
222
223 plt.figure(figsize=(15, 8))
224 plt.subplot(2, 1, 1)
225 plt.plot(ep_length_history, label='Len')
226 plt.grid(True)
227 plt.subplot(2, 1, 2)
228 plt.plot(self.metrics['episode_returns'], label='Return')
229 plt.grid(True)
230 plt.legend()
231 plt.tight_layout()
232 plt.savefig(os.path.join(results_dir, 'td3_lqr_train_episode_returns.png'), dpi=200,
    bbox_inches='tight')
233 plt.show()
234
235 return self.metrics
236
237 if __name__ == '__main__':
238     from scripts.Quadrotor3D import Quadrotor3D
239     from scripts.QuadrotorTrajectories import QuadrotorTrajectory
240     import copy
241
242     results_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'results
    ', 'p3'))
243     os.makedirs(results_dir, exist_ok=True)
244
245     # Known parameters
246     known_params = {
247         'm': 0.027, # kg
248         'L': 0.0397, # m
249         'g': 9.8, # m/s
250     }
251
252     # Unknown parameters
253     unknown_params = {
254         'Ixx': [0.6e-5, 3.0e-5],
255         'Iyy': [0.6e-5, 3.0e-5],
256         'Izz': [1.0e-5, 5.0e-5],
257         'KF': [2.0e-10, 4.5e-10],
258         'KM': [4.0e-12, 1.3e-11],
259     }
260
261     # System
262     Ts = 0.01
263     system = Quadrotor3D(Ts=Ts)
264
265     # Trajectory
266     trajectory_gen = QuadrotorTrajectory(sample_time=Ts)
267     LENGTH = 12
268     references_raw = trajectory_gen.helix(length=LENGTH, num_cycles=3, radius=1, rise
    =0.5, center=(0., 0., 0.5))
269     references = transform_references(references_raw)
270
271

```

```

272
273 # Controller
274 Q_lqr = np.diag([
275     5., 2., # x,
276     5., 2., # y,
277     5., 2., # z,
278     1., 1., 0.1, # , ,
279     0.1, 0.1, 0.1, # p, q, r
280 ])
281 R_lqr = np.eye(4) * 0.01
282 rng = np.random.default_rng(seed=42)
283 unk_params = {k: rng.uniform(low=v[0], high=v[1]) for k, v in unknown_params.items()}
284
285 # LQR has a wrong fixed model
286 lqr_model_params = known_params.copy()
287 lqr_model_params.update(unk_params)
288 lqr_internal_model = Quadrotor3D(Ts=Ts, params=lqr_model_params)
289 u_min = 0
290 n = len(lqr_internal_model.x)
291 m = 4
292 u_max = (lqr_internal_model.p['m'] * lqr_internal_model.p['g'] / 4.0) * 2.5
293 lqr = LQR(lqr_internal_model, n, m, Q_lqr, R_lqr, u_lims=[u_min, u_max])
294
295 ## AGENTE TD3
296 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
297
298 agent = TD3Agent(
299     reward_function=None, # se calcula externa con el del LQR
300     obs_dim=obs_dim,
301     act_dim=act_dim,
302     act_low=act_low,
303     act_high=act_high,
304     cfg=TD3Config(),
305     device=device,
306     max_pos_error=2.0,
307     max_angle_deg=60.0,
308 )
309 unknown_params_gaussian = {
310     k: [(v[1]+v[0])/2.0, abs((v[1]+v[0])/2.0 - v[0])/2.0]
311     for k, v in unknown_params.items()
312 }
313
314 ## ENTRENAMOS!!
315 best_model_path = os.path.join(results_dir, 'best_td3_actor.pt')
316 train_ans = input('Entrenar?[s/N]:').strip().lower()
317 do_train = train_ans in ('s', 'si', 's ', 'y', 'yes', '1', 'true')
318
319 if do_train:
320     train(agent, copy.deepcopy(system), lqr, references,
321           known_params, unknown_params_gaussian,
322           seed=0, steps=100_000, eval_every=1_000)
323
324 if not os.path.exists(best_model_path):
325     raise FileNotFoundError(
326         f"No existe el modelo guardado."
327     )
328
329 agent.actor.load_state_dict(

```

```

330         torch.load(best_model_path, map_location=device))
331     agent.actor.eval()
332
333     # Simulation
334     N_steps = references.shape[1]
335     x_current = references[:, 0]
336     system.x = list(x_current)
337     f_hover = known_params['m'] * known_params['g'] / 4.0
338     last_u = np.ones(4)*f_hover
339
340     x_list = []
341     x_est_list = []
342     u_list = []
343     y_list = []
344     total_reward = 0
345     total_reward_simp = 0
346     init_time = time.time()
347
348     rmse_inst_list = []
349     pos_sq_err_hist = []
350
351     for k in range(N_steps):
352         # Extract the reference
353         ref = references[:, k]
354
355         # Get LQR action
356         u_opt = lqr(x_current, ref)
357
358         # Step the nonlinear system
359         _, y = system.step(u_opt)
360
361
362         total_reward += reward_function_complex(ref, x_current, u_opt, last_u, f_hover)
363         total_reward_simp += reward_function_simple(ref, x_current, u_opt)
364
365
366         x_current = np.array(system.x)
367         pos_err = x_current[[0, 2, 4]] - ref[[0, 2, 4]]
368         pos_sq_err = pos_err ** 2
369         pos_sq_err_hist.append(pos_sq_err)
370
371         rmse_inst = float(np.sqrt(np.mean(pos_sq_err)))
372         rmse_inst_list.append(rmse_inst)
373
374         rmse_cum = float(np.sqrt(np.mean(np.array(pos_sq_err_hist))))
375         x_list.append(np.array(system.x).reshape(-1, 1))
376         y_list.append(np.array(y).reshape(-1, 1))
377         u_list.append(np.array(u_opt).reshape(-1, 1))
378
379         last_u = u_opt
380
381         if k % 100 == 0:
382             print('Step{}|{}|RMSE_inst(pos): {:.4f}|RMSE_cum(pos): {:.4f}'.
383                   format(k, N_steps, rmse_inst, rmse_cum))
384
385     print('Total_time_LQR: {}'.format(time.time() - init_time))
386     print('Total_reward_LQR: {}'.format(total_reward))
387     print('Total_reward_Simp_LQR: {}'.format(total_reward_simp))

```



```

388 x_list = np.concatenate(x_list, axis=1)
389 y_list = np.concatenate(y_list, axis=1)
390 u_list = np.concatenate(u_list, axis=1)
391 print(x_list.shape, references.shape)
392
393 pos_err_all = x_list[[0, 2, 4], :] - references[[0, 2, 4], :]
394 rmse_xyz = np.sqrt(np.mean(pos_err_all**2, axis=1))
395 rmse_pos = np.sqrt(np.mean(pos_err_all**2))
396 print('RMSE final x,y,z [m] [LQR]={}'.format(rmse_xyz))
397 print('RMSE final posicion global [m] [LQR]={}'.format(rmse_pos))
398
399
400 ### LOOP TD3 con LQR:
401 system_TD3 = Quadrotor3D(Ts=Ts)
402 x_current_td3 = references[:, 0].copy()
403 system_TD3.x = list(x_current_td3)
404
405 lqr_td3 = copy.deepcopy(lqr)
406 prev_action = None
407 prev_u_td3 = None
408 action = np.ones(act_dim)
409 obs = build_obs_tuning(x_current_td3, references[:, 0], system_TD3.obs_norm)
410
411 x_list_td3, u_list_td3 = [], []
412 q_diag_td3_hist, r_diag_td3_hist = [], []
413 total_reward_td3 = 0.0
414 pos_sq_err_hist_td3 = []
415
416 for K in range(N_steps):
417     ref = references[:, K]
418
419     action = agent.act(torch.from_numpy(obs).float().to(device), noise_scale=0.0)
420     Q_td3_k, R_td3_k = apply_action_to_lqr(lqr_td3, action, Q_lqr, R_lqr)
421     u_opt_td3 = lqr_td3(x_current_td3, ref)
422
423     system_TD3.step(u_opt_td3)
424     x_current_td3 = np.array(system_TD3.x)
425
426     total_reward_td3 += reward_function_complex(ref, x_current_td3, u_opt_td3,
427         prev_u_td3, f_hover)
428
429     pos_sq_err_hist_td3.append((x_current_td3[[0, 2, 4]] - ref[[0, 2, 4]]) ** 2)
430     x_list_td3.append(x_current_td3.copy())
431     u_list_td3.append(u_opt_td3.copy())
432     q_diag_td3_hist.append(np.diag(Q_td3_k).copy())
433     r_diag_td3_hist.append(np.diag(R_td3_k).copy())
434
435     obs = build_obs_tuning(x_current_td3, ref, system_TD3.obs_norm)
436     prev_action = action
437     prev_u_td3 = u_opt_td3
438
439 # estados y acciones al sistema (no del TD3)
440 x_list_td3 = np.array(x_list_td3).T
441 u_list_td3 = np.array(u_list_td3).T
442
443 pos_err_td3 = x_list_td3[[0, 2, 4], :] - references[[0, 2, 4], :]
444 rmse_td3 = np.sqrt(np.mean(pos_err_td3**2))
445
446 ### print de resultados TD3+LQR

```

```

446 print('Total_reward_TD3+LQR:{}'.format(total_reward_td3))
447 print('RMSE_final_posicion_global_{m}_TD3+LQR={:.4f}_{m}'.format(rmse_td3))
448 print('Total_reward_SOLO_LQR:{}'.format(total_reward))
449 print('RMSE_final_posicion_global_{m}_SOLO_LQR={:.4f}_{m}'.format(rmse_pos))
450
451 q_diag_td3_hist = np.array(q_diag_td3_hist)
452 r_diag_td3_hist = np.array(r_diag_td3_hist)
453 q_diag_final = q_diag_td3_hist[-1]
454 r_diag_final = r_diag_td3_hist[-1]
455 q_diag_mean = np.mean(q_diag_td3_hist, axis=0)
456 r_diag_mean = np.mean(r_diag_td3_hist, axis=0)
457
458 np.set_printoptions(precision=4, suppress=True)
459 print('\n==_Matrices_LQR_aprendidas_por_TD3==')
460 print('diag(Q)_base_{}={}'.format(np.diag(Q_lqr)))
461 print('diag(Q)_TD3_final_{}={}'.format(q_diag_final))
462 print('diag(Q)_TD3_promedio_{}={}'.format(q_diag_mean))
463 print('escala_Q_final/base_{}={}'.format(q_diag_final / np.diag(Q_lqr)))
464 print('Q_final_{}_{}_base?_{}={}'.format(q_diag_final, np.diag(Q_lqr), atol=1e-8))
465
466 print('diag(R)_base_{}={}'.format(np.diag(R_lqr)))
467 print('diag(R)_TD3_final_{}={}'.format(r_diag_final))
468 print('diag(R)_TD3_promedio_{}={}'.format(r_diag_mean))
469 print('escala_R_final/base_{}={}'.format(r_diag_final / np.diag(R_lqr)))
470 print('R_final_{}_{}_base?_{}={}'.format(r_diag_final, np.diag(R_lqr), atol=1e-8))
471
472 plt.figure(figsize=(15, 7))
473 plt.subplot(4, 1, 1)
474 plt.plot(references[0, :], 'k--', linewidth=2, label='Reference')
475 plt.plot(x_list[0, :], label='LQR', linewidth=1.5)
476 plt.plot(x_list_td3[0, :], label='TD3+LQR', linewidth=1.5, alpha=0.8)
477 plt.grid(True)
478 plt.legend()
479 plt.ylabel('X_{m}')
480
481 plt.subplot(4, 1, 2)
482 plt.plot(references[2, :], 'k--', linewidth=2, label='Reference')
483 plt.plot(x_list[2, :], label='LQR', linewidth=1.5)
484 plt.plot(x_list_td3[2, :], label='TD3+LQR', linewidth=1.5, alpha=0.8)
485 plt.grid(True)
486 plt.legend()
487 plt.ylabel('Y_{m}')
488
489 plt.subplot(4, 1, 3)
490 plt.plot(references[4, :], 'k--', linewidth=2, label='Reference')
491 plt.plot(x_list[4, :], label='LQR', linewidth=1.5)
492 plt.plot(x_list_td3[4, :], label='TD3+LQR', linewidth=1.5, alpha=0.8)
493 plt.grid(True)
494 plt.legend()
495 plt.ylabel('Z_{m}')
496
497 plt.subplot(4, 1, 4)
498 plt.plot(u_list[0, :], label='LQR_u1', linewidth=1.5)
499 plt.plot(u_list_td3[0, :], label='TD3+LQR_u1', linewidth=1.5, alpha=0.8)
500 plt.plot(u_list[1, :], label='LQR_u2', linewidth=1.5)
501 plt.plot(u_list_td3[1, :], label='TD3+LQR_u2', linewidth=1.5, alpha=0.8)
502 plt.grid(True)
503 plt.legend(fontsize=8)
504 plt.ylabel('Control_{N}')
```

```

505 plt.tight_layout()
506 plt.savefig(os.path.join(results_dir, 'lqr_vs_td3_lqr_tracking_xyz_u.png'), dpi=200,
507             bbox_inches='tight')
508 plt.show()
509
510 fig = plt.figure(figsize=(12, 8))
511 ax = fig.add_subplot(111, projection='3d')
512
513 ax.plot(references[0, :], references[2, :], references[4, :],
514         color='k', linewidth=2.5, linestyle='--', label='Reference')
515 ax.plot(x_list[0, :], x_list[2, :], x_list[4, :],
516         color='tomato', linewidth=2, label='LQR_trajectory')
517 ax.plot(x_list_td3[0, :], x_list_td3[2, :], x_list_td3[4, :],
518         color='limegreen', linewidth=2, label='TD3+LQR_trajectory', linestyle=':')
519
520 ax.set_xlabel('X[m]')
521 ax.set_ylabel('Y[m]')
522 ax.set_zlabel('Z[m]')
523 ax.set_title('Quadrotor Trajectories - Reference vs LQR vs TD3+LQR')
524 ax.legend(fontsize=10)
525 plt.tight_layout()
526 plt.savefig(os.path.join(results_dir, 'lqr_vs_td3_lqr_trajectory_3d_comparison.png'),
527             dpi=200, bbox_inches='tight')
528 plt.show()

```